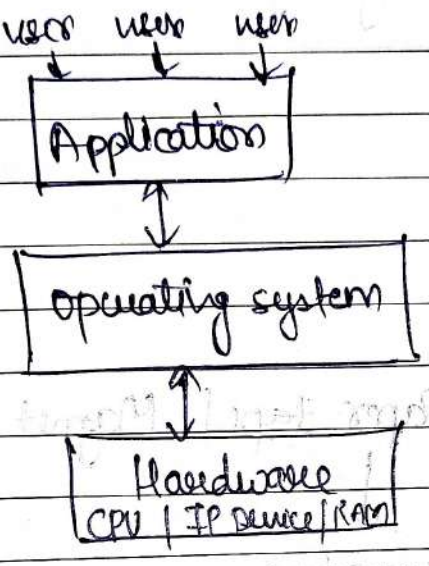
Date 5-08-24

Date / / Page No. Shivalal

OPERATING SYSTEM

⊙ Operating System :-

⊙ Operating system is a system software, It work as a interface between user and hardware.



Ex- windows, Linux etc.

⊙ Primary Goal | Functions :-

1. Convinence

(i) Resource management.
(ii) Process management.
(iii) Storage management.
(iv) Memory management.
(v) Security
(vi) Booting.
(vii) Multitasking.
(viii) File management
(ix) Platform for other application software.

⊙ Primary goal :-
(i) manage hardware.
(ii) user-friendly interface.
(iii)

⊙ Secondary goal :-
(i) System security.
(ii) maintain reliability and stability.

⊙ Types of operating System:-

1. Batch.
2. Multi puggrammed.
3. Multitasking.
4. Real time operating system.
5. Distributed.
6. clustered
7. Embedded.

(i) Batch:-

⊙ Punch cards | Paper tape | Magnet tape.

↓

operator.

↓ distribute into batches.

B_1 B_2 B_2

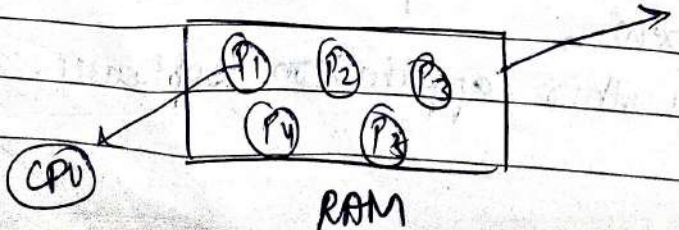
↓

CPU.

In a Batch operating system, the similar jobs are grouped together into batches with the helps of some operator and these batches are executed one by one.

(ii) Multi puggrammed O.S:-

⊙ Non-preemptive.



Date / /
Page No.
Shivalal

(i) Maximize CPU utilization.

(ii) Multiprogramming means more than one process in Main memory which are ready to execute.

(iii) CPU never idle unless, there is not process ready to execute or at time of context switch.

(iii) Multitasking O.S:-

→ Multitasking is Multiprogramming with time-sharing.

→ Single CPU.

→ Context switching.

→ Time-sharing.

→ A multitasking operating system allows multiple tasks to run at the same time, sharing resources like the central processing unit (CPU) and main memory.

(iv) Real time operating system:-

⊙ A Real time O.S is an O.S that guarantees real-time applicati a certain capability with a specified deadline.

→ RTOS processing time requirements are measured in milliseconds.

Ex- Air bags, remote etc.

→ Types of RTOS:-

1. Hard-RTOS.
2. Soft-RTOS.

(v) Distributed operating system:-

⊙ A distributed operating system is a type of O.S that connects multiple computer systems to work together to serve application.

Each system in a distributed O.S is an independent unit that can have its own operating system, applications and data.

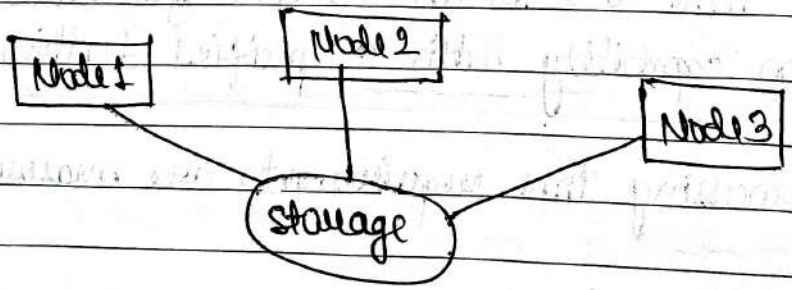
Note:- Also called loosely coupled system.

→ Types:-

1. client-server system.
2. peer-to-peer system.

(vi) clustered operating system:-

⊙ A group of computers connected in a local area network is called a cluster system. In a clustered operating system the computers, or CPUs share single storage, and they all cooperate to perform each task at once.



(vii) Embedded operating system:-

⊙ An embedded operating system is a specialized operating

Date / /
Page No.
Shivalal

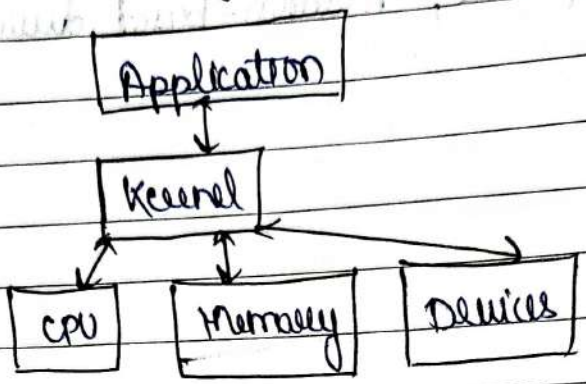
system designed to perform a specified task for a device that is not a computer. The main job of an embedded OS is to run the code that allows the device to do its job.

Ex. ATMs, cellphones, Arduino-based devices, etc.

Operating System Structures:

⊙ Kernel:-

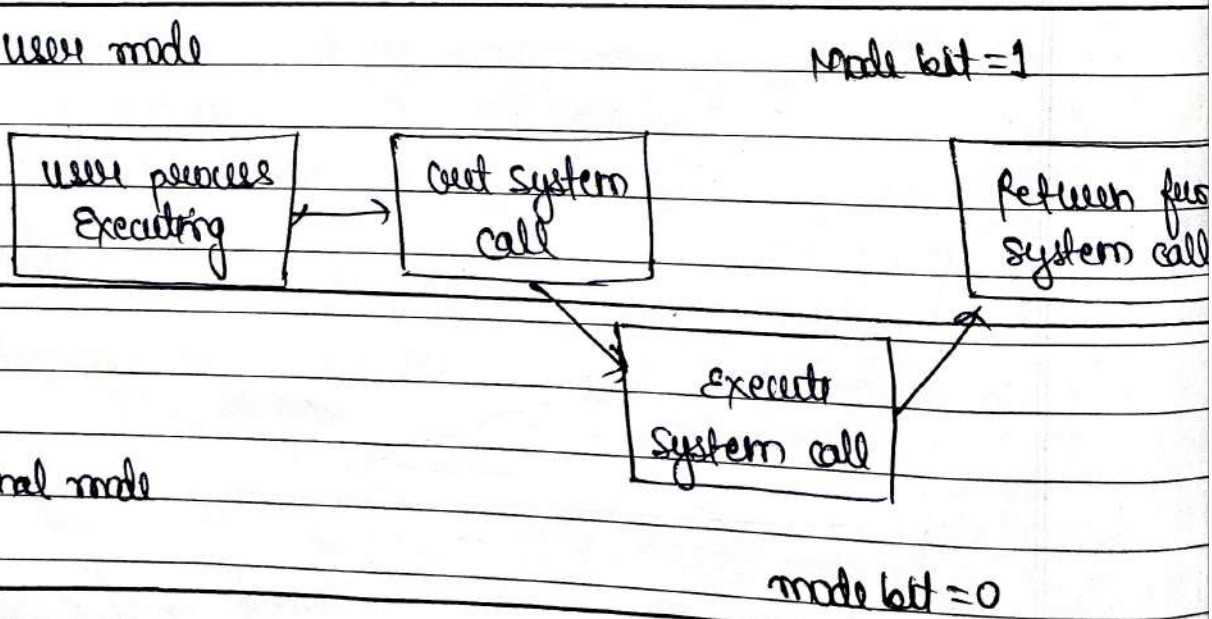
⊙ It is a computer program that is core of OS, with complete control over everything in system.



→ mode bit = 1 for user mode
→ mode bit = 0 for kernel mode

⊙ User mode vs Kernal mode:-

Note:- To excess the kernal we used system call.



⊙ It is an interface between user application and hardware.

⊙ Types of kernal in operating system:-

System call:-

(i) File related:-

→ open()
→ read()
→ write()
→ close()
→ create file() etc.

(ii) Device related:-

→ Read
→ write
→ Reposition
→ ioctl
→ fcntl

(iii) Information:-

→ get pid
→ attributes
→ get system time and date, data.

(iv) process control:-

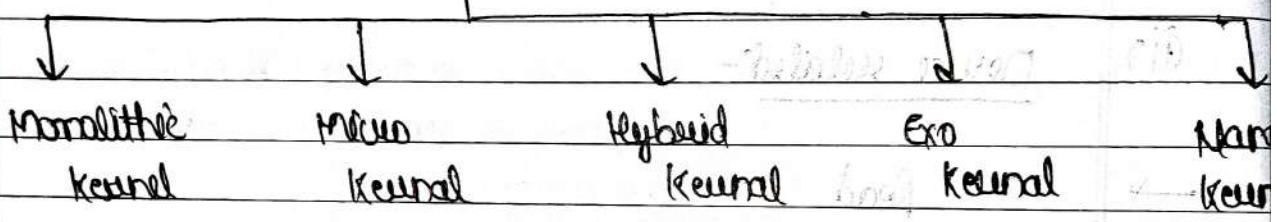
→ load
→ Execute
→ about
→ fork

→ wait
→ Signal
→ Allocate etc.

(v) Communication:-

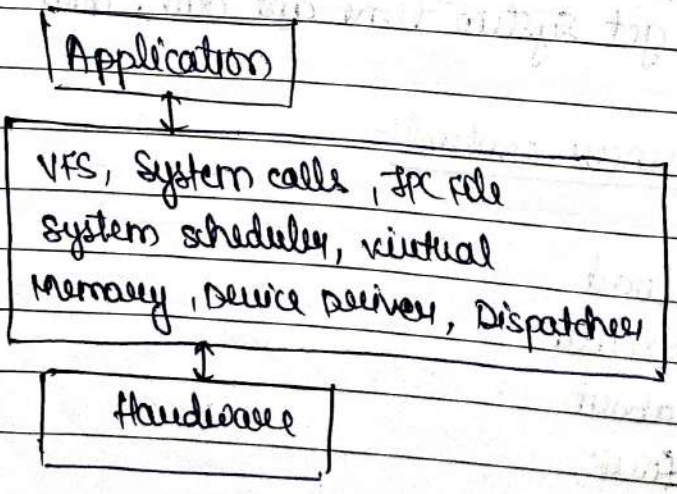
→ Pipe()
→ Create / delete Connections.
→ shmget().

1. Types



(i) Monolithic Kernel:-

• A monolithic kernel is a operating system kernel in which all the operating system services run in kernel space, meaning they all share the same memory space. This type of kernel is characterized by its tight integration of system services and its high performance.



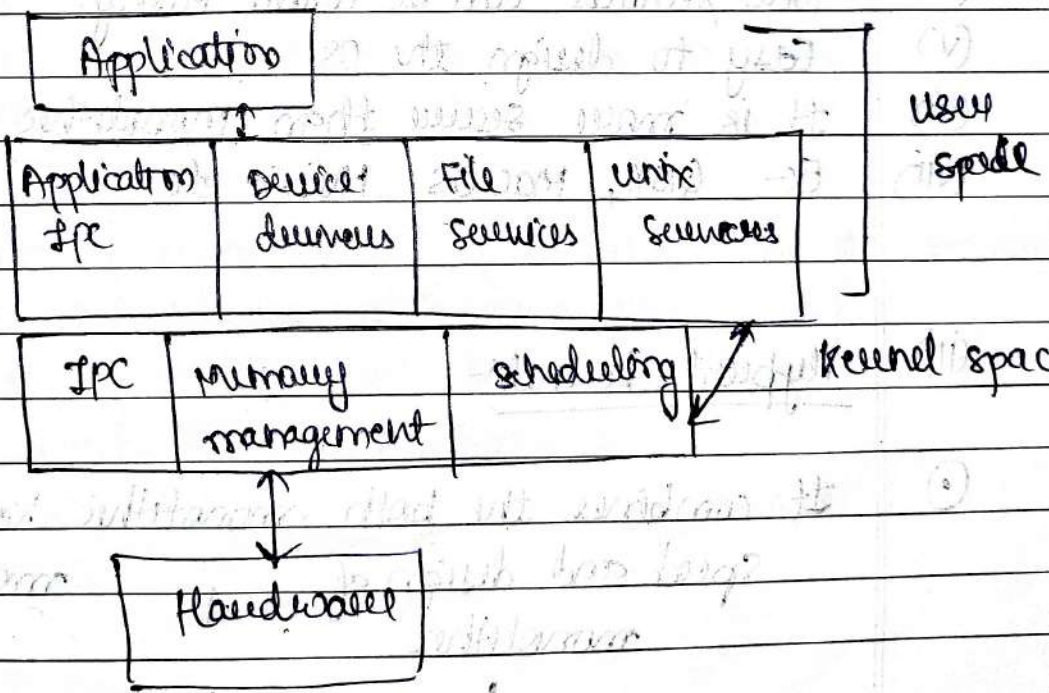
→ If a service crashes, the whole os crashes

(ii) Micro-kernel:-

⊙ It provides the most basic service, required for an operating system to function such as memory management and process scheduling.

Other services such as device drivers and file systems, are implemented as user-level process that communicate with the microkernel via message passing.

Essential functions
- Interprocess communication (IPC)
- Memory management
- Scheduling.



Difference between Monolithic kernel & microkernel:-

(i) Monolithic kernel:-

(i) It is larger in size but fast in execution.
(ii) These kernel are difficult to debug.
(iii) If user has to add new services, used need to modify the entire OS.
(iv) Security issue exist always, as there is no isolation among various services present in the kernel.
(v) Fastest in execution. / complex to design.
(vi) Ex- DOS, Unix, Solaris, Linux., etc.

(ii) **Micro kernel:-**

(i) It is smaller in size.
(ii) Slower in execution
(iii) Easy to debug.
(iv) New services can be added easily.
(v) Easy to design the OS.
(vi) It is more secure than monolithic kernel.
(vii) Ex- QNX, Mac OS, Minix etc.

(iii) **Hybrid kernel:-**

⊙ It combines the both monolithic kernel and microkernel

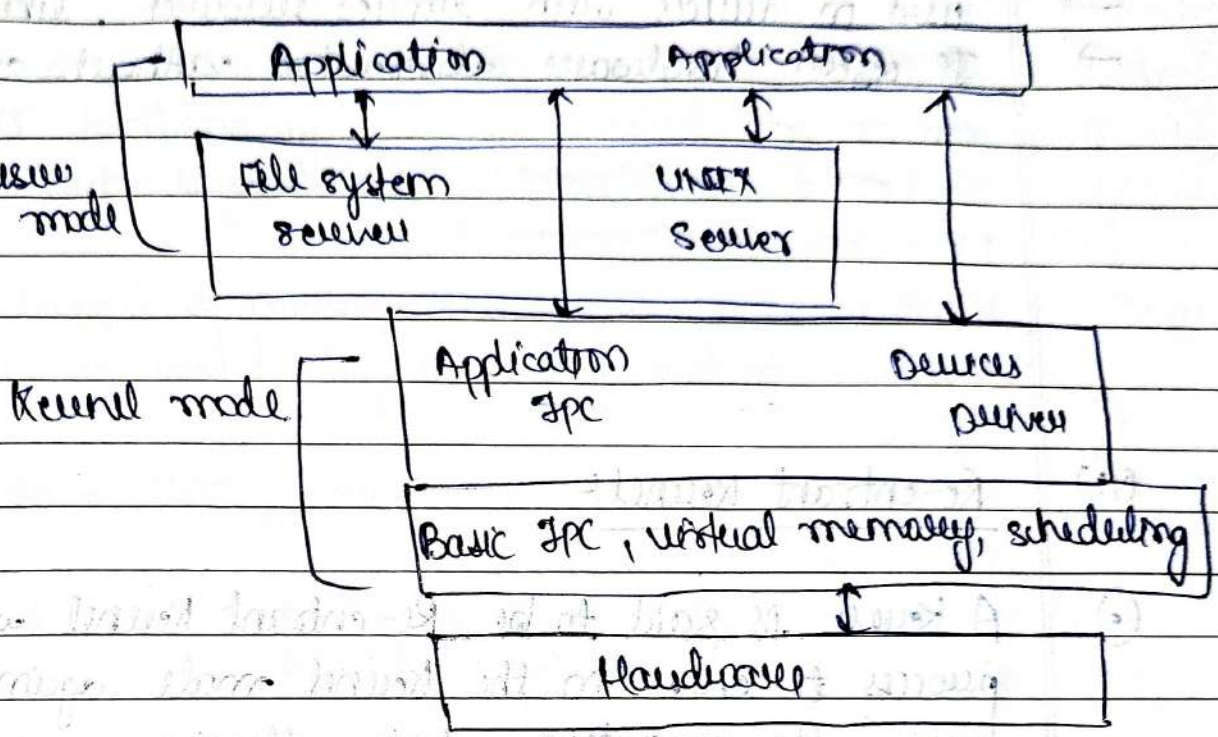
Speed and design of monolithic + modularity and stability of microkernel

Hybrid kernel.

Note:- Still similar to a monolithic kernel.

Date / /
Page No.
Shivalal

→ Hybrid kernel are offer better performance than microkernel. because they reduce the number of context switches required between user space and kernel space.



(iv) EXO-Kernel :-

⊙ It is the type of kernel which follow end-to-end principle
It has fewest hardware abstraction as possible.
→ Lets applications directly control hardware.
→ promotes flexibility and efficiency.
→ Ex- Nemesis (Research) and EXOS (experiment)

Diagram

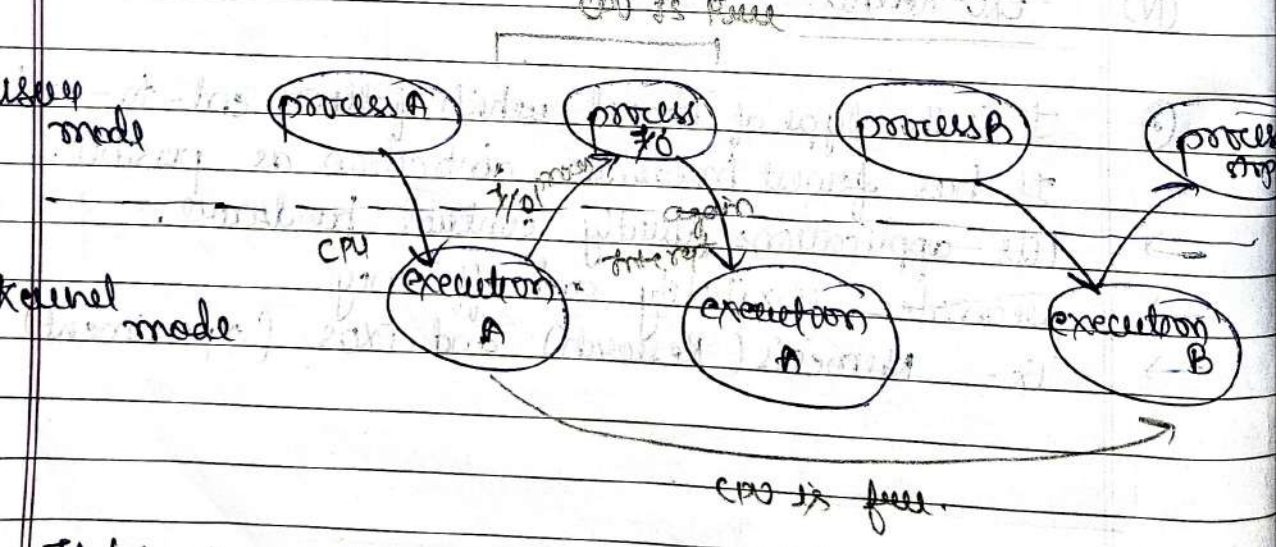
(v) NANO Kernel :-

⊙ Smallest kernel : a few thousand lines of code.
→ used in devices with limited resources, like IOT gadgets
→ It offers hardware abstraction without system services.

Diagram

(vi) Re-entrant Kernel :-

⊙ A kernel is said to be Re-entrant kernel which allows a process to enter in the kernel mode again & again during its execution, which allowing execution of another process in kernel mode at the same time.



⊙ It helps in multiprocessing.

Date / /
Page No.
Shivalal

System structured :-

⊙ Layered structure :-

⊙ Layered structured os is the operating system in which the whole functionality of os is divided into a no. of layers with each layer having a well defined specific functions.

* The concept of arranging different tasks of os into different layers is called "layered structured of os".

Ex- windows, UNIX, Linux etc.

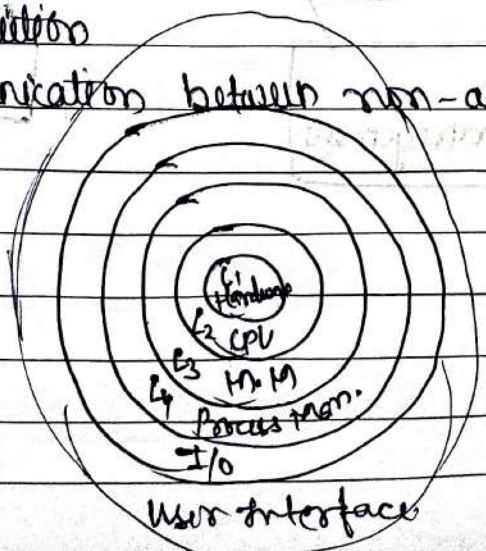
Note:- Exact number of layers depends upon specific O.S.

→ Advantages:-

(i) Easy debugging
(ii) Easy updati
(iii) No direct access to hardware.

→ Disadvantages:-

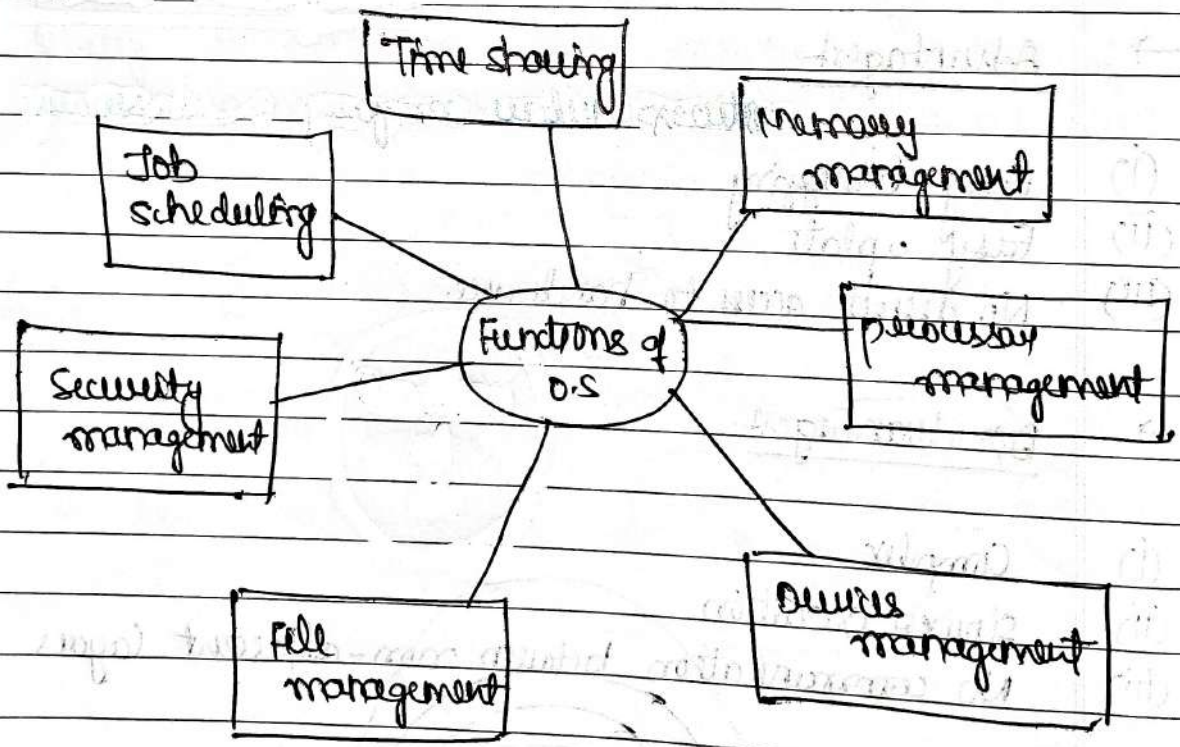
(i) Complex
(ii) Slower execution
(iii) No communication between non-adjacent layers



Date / /
Page No.
Shivalal

⊙ Operating system components

(b) Process management.
(a) Files management.
(3) System calls
(4) Network management.
(5) I/O device management.
(c) Main memory Management
(7) Secondary memory / storage management.
(8) Security management.
(9) Signals
(10) Job accounting
(11) Error detecting.



Date / /
Page No.
Shivalal

1. Memory management:-

- Memory management module performs the task of allocation and de-allocations of memory space to programs in need to this resources.

2. Process management:-

- The operating system assigns processors to the different tasks that must be performed by the computer system.

3. Device Management:-

- operating system performs the task of allocations and de-allocation of the devices.

4. File management:-

- operating system manages all the file-related activities such as organization storage, retrieval, naming, sharing and protection of files.

5. Security management:-

- Security management functions of an operating system helps in implementing mechanism that secure and protect the computer system internally as well as externally.

6. Job scheduling:-

- Job scheduling is the process of allocating system resources

to many different tasks by an operating system.

(7) Time sharing:-

(•) It co-ordinates and assigns compilers, assemblers, utili
programs, and other software packages to various
user working on computer system.

Date / /
Page No.
Shivalal

Threads and their management

⊙ Threads:-

⊙ A thread is the smallest unit of execution within a process.
Multiple threads can exist with a single process, sharing resources but executing independently.

→ Imagine a thread as a mini-job inside a big job (the process). A big job can have multiple mini-jobs running at the same time. Each mini-job (thread) can do its own thing without waiting for the others.

→ Types of threads:-

1. User-level Threads.
2. Kernel-level threads.
3. Hybrid Threads.

→ Advantages of threads:-

1. Faster Responses
2. sharing Resources.
3. Better performance.
4. Quicker communication
5. Smarter use of CPU
6. Easier programming.

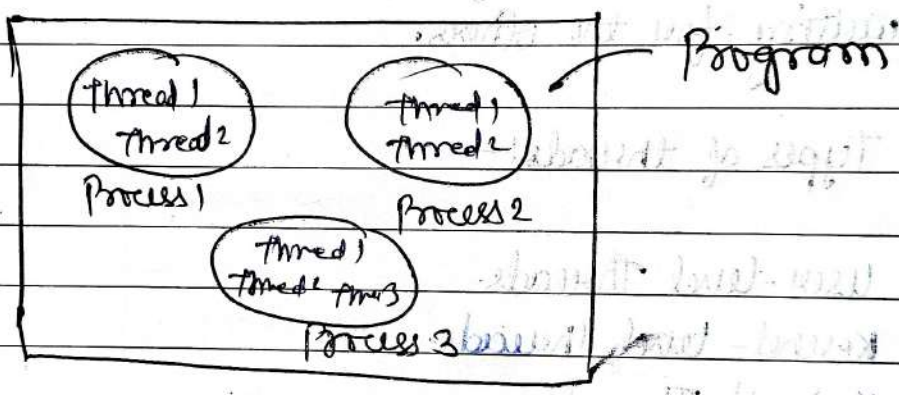
Date / /
Page No.
Shivalal

Process :-

⊙ Process :-

⊙ process is a program under execution. In the operating system, a process is something that is currently under execution. So, an active program can be called a process.

Ex :- When you want to search something on web then you start a browser, so this can be a process.



(i) Instruction :- A line of code.

(ii) Program :- Group of instruction.

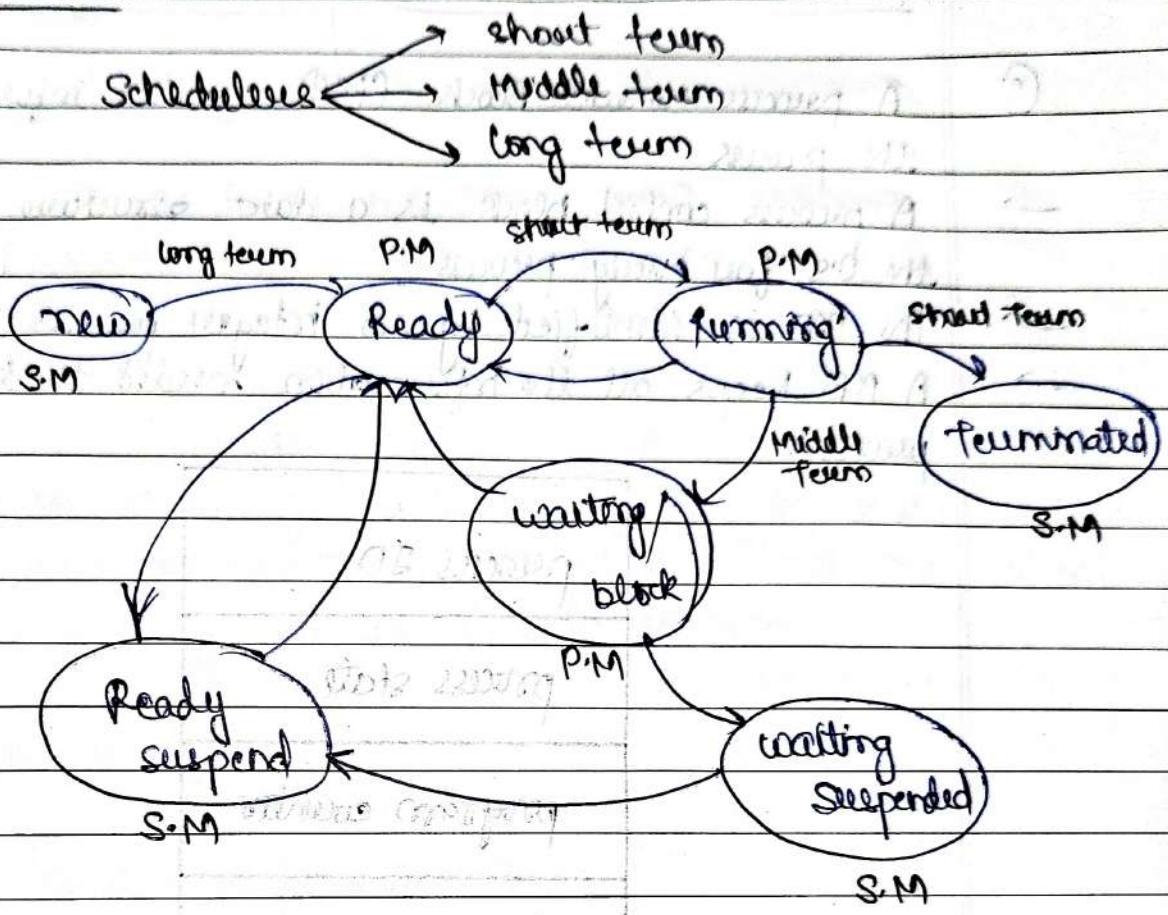
(iii) process :- program in execution.

(iv) Threads :- It is a light weight process. | Smallest unit of program | In process we have multiple threads.

→ degree of M.P = no. of process in Ready state

Date / /
Page No.
Shivalal

⊙ **Process state :-**



(i) P.M → Primary memory

(ii) S.M → Secondary memory

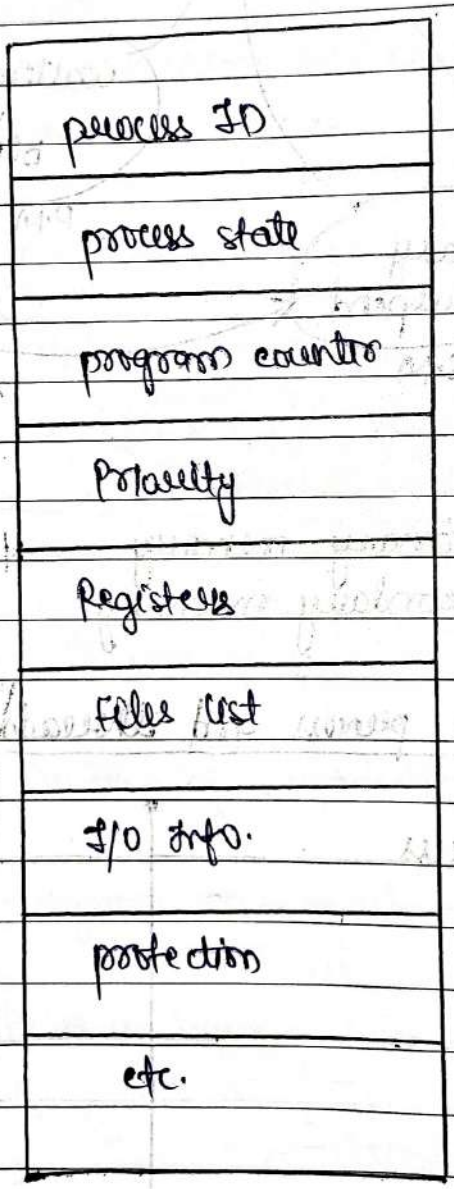
⊙ Process control block (PCB) :-

⊙ A process control block (PCB) contains information about the process.

→ A process control block is a data structure maintained by the O.S for every process.

→ The PCB is identified by an integer process ID (PID)

→ A PCB keeps all the information needed to keep track on a process.



⊙ scheduling Algorithms:-

(i) Pre-Emptive :-

⇒ SRTF (Shortest Remaining time first).
→ LRTF (Longest Remaining time first) (extra)
⇒ Round Robin.
⇒ Priority based.

⊙ preemptive scheduling is a method of assigning tasks based on their priorities, where a higher priority task can interrupt a lower priority task is already running.

(ii) Non- preemptive :-

⇒ FCFS (First come first serve)
⇒ SJF (shortest Job first).
→ LJF (Longest Job first) (extra)
→ HRRN (Highest Response Ratio Next) (extra)
⇒ Multilevel Queue

⊙ Non-preemptive scheduling is a CPU scheduling method where a process keeps using a resource until it finishes its task or voluntarily gives us control of the CPU.

⊙ CPU Scheduling :-

(i) Arrival time :-

⊙ The time at which process enter the Ready Queue or state

(ii) Burst time:-

⊙ Time required by process to get execute on CPU.

(iii) Completion time:-

⊙ The time at which process complete its execution.

(iv) Turn Around time:-

⊙ {Completion time − Arrival time}

(v) Waiting time:-

⊙ {Turn around time − Burst time} [CPU Bound / I/O Bound]

(vi) Response time:-

⊙ {(The time at which a process get CPU first time) − (Arrival time)}

CPU scheduling in os is a method by which one process is allowed to use the CPU while the other processes are kept on hold. all are kept in the waiting state.

1. FCFS (first come first serve):-

⊙

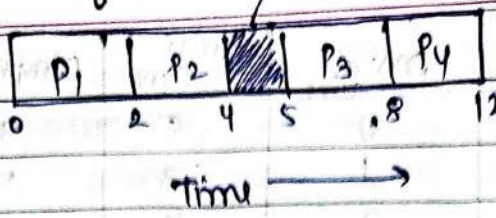
process no.	Arrival time	Burst time	Completion time	TAT	WT	RT
P_1	0	2	2	2	0	0
P_2	1	2	4	3	1	1
P_3	5	3	8	3	0	0
P_4	6	4	12	6	2	2

Disadvantage: Convoy effect (problem)

(i) Avg Turn Around Time
(ii) Avg waiting time.
(iii) Context Switching.

Date / /
Page No.
Shivalal

Gantt chart (Gantt)

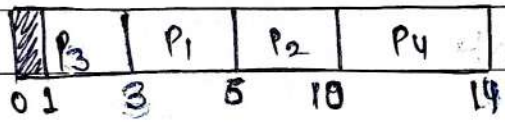


Mode :- Non-preemptive
criteria :- Arrival time

2. SJF (Shortest Job first) :-

process no.	Arrival time	Burst time	Complete time	TAT	WT	RT
P_1	1	3	6	5	2	2
P_2	2	4	10	8	4	4
P_3	1	2	3	2	0	0
P_4	4	4	14	10	6	6

Gantt chart



mode :- Non-preemptive
criteria :- Burst time

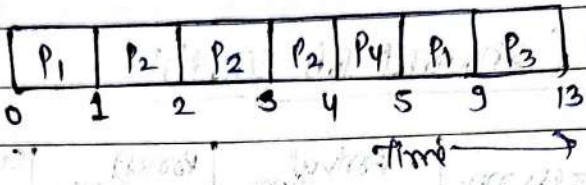
3. SRTF (Shortest Remaining time first) :-

⊙ The SJF shortest job first with + preemptive mode is known as SRTF (shortest Remaining time first).

Date / /
Page No.
Shivalal

process no.	Arrival time	Burst time	completion time	TAT	WT	RT
P_1	0	5	9	9	4	0
P_2	1	3	4	3	0	0
P_3	2	4	13	11	7	7
P_4	4	1	5	1	0	0

Gantt chart



Note:- Criteria :- 'Burst time'
Mode :- 'Preemptive'

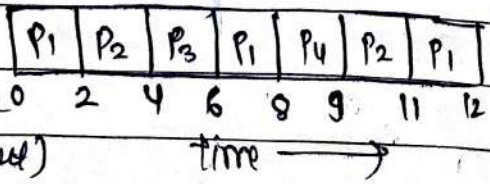
4. Round Robin:-

Process no.	Arrival time	Burst time	completion time	TAT	WT	RT
P_1	0	5	12	12	7	0
P_2	1	4	11	10	6	1
P_3	2	2	6	4	2	2
P_4	4	1	9	5	4	4

Given:- Time quantum = 2

Ready queue

Running queue (Gantt chart)



Criteria :- Time Quantum
Mode :- preemptive

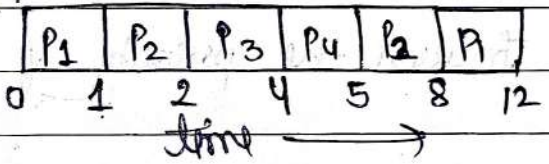
5. Priority scheduling :-

⊙

priority	Process no	A.T	B.T	C.T	T.A.T	W.T	R.T
10	P_1	0	5	12	12	7	0
20	P_2	1	4	8	7	3	0
30	P_3	2	2	4	6	4	0
40	P_4	4	1	5	1	0	0

Note :- Higher the no. higher the priority.

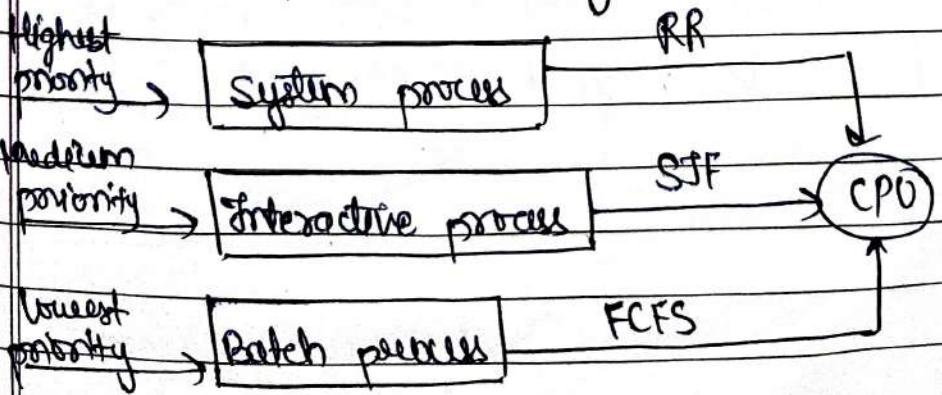
→ Gantt chart :-



criteria = 'priority'
mode = 'preemptive'

→ A.W.T = 2.5
→ A.T.A.T = 5.5

⑥ Multilevel queue scheduling :-



⊙ In MLQ process are divided into multiple queues based on

three priority, with each queue having a different priority level. Higher-priority processes are placed in queues with higher priority levels, while those are placed with lower priority level priorities are divided on the basis of type, char. and importance.

Note:

⑦ Multilevel Feedback scheduling:-

⊙ Multilevel Feedback queue scheduling (MLFQ) is like Multilevel queue scheduling but in this process can move between the queue. And thus, much more efficient than multilevel queue scheduling.

Note:- It also remove the problem of starvation.

Note:- Aging:- Increase the priority of process (lowest process)

Question

Multiprocessor - scheduling

⊙ In multiple-processor scheduling, multiple CPUs are available
→ load sharing become possible as distributed among these available processor.

Multiple processor scheduling is more complex as compared to single processor scheduling.

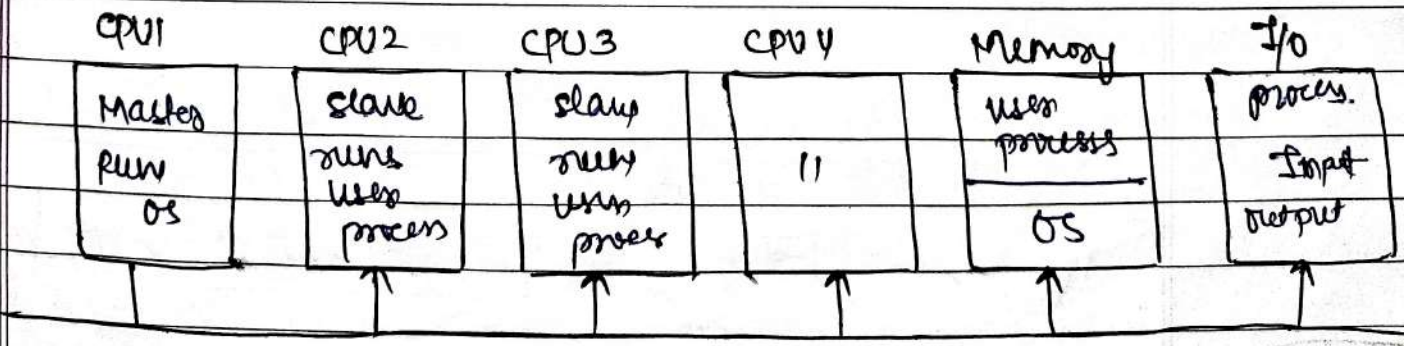
→ It is of two types:-

1. Asymmetric scheduling.
2. Symmetric scheduling.

1. Asymmetric scheduling:-

⊙ In this scheduling approach, all scheduling decision, I/O processing and resource allocation are handled by a single processor which is called the master processor and the other processor executes only the user code.

→ This is simple and reduces the need of data sharing.
→ this entire scenario is called Asymmetric Multiprocessing.

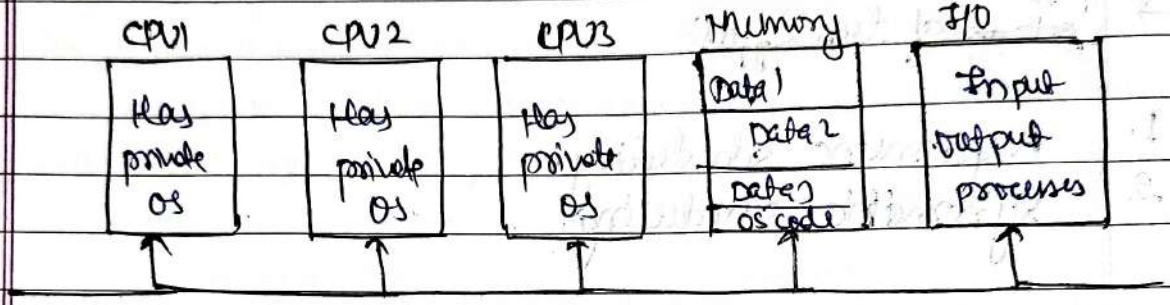


2. Symmetric scheduling

Date / /
Page No.
Shivalal

⊙ A second approach uses Symmetric Multiprocessing where each processor is self scheduling. All processes may be in the common ready queue or in its own private queue.

→ The scheduling proceeds further by having the scheduling for each processor examine the ready queue and select a process to execute.



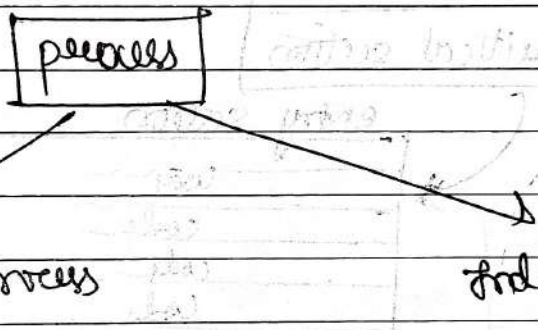
⊙ Critical section:-

Process synchronization:-

⊙ Process synchronization is a fundamental concept in operating system (OS) that manages and coordinates multiple processes that share resources and data. It's important for ensuring that processes run smoothly and without conflict.
∴(same resource)

⊙ process synchronization helps to:-

1. Maintain data consistency.
2. Allocate resource fairly.
3. Establish execution order.



1. Cooperative process:- In OS, Cooperative process are the process that can affect or be affected by other process that are running.

Ex:- int shared = 5
∴(uniprocessor)

P_1	P_2
int x = shared;	int y = shared;
x++;	y--;
sleep(1);	sleep(1);
shared = x;	shared = y;

pause for 1 second

(race condition) ∴ problem #

Date / /
Page No.
Shivalal

⊙ principle of concurrency:-

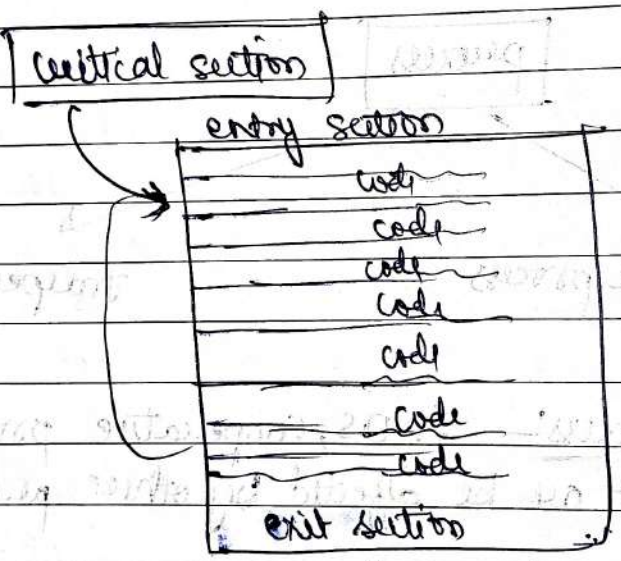
⊙ Concurrency is the execution of the multiple instruction sequences at the same time.

→ It happens in the os when there are several process threads running in parallel.

→ The processes communicate through shared memory or message passing.

⊙ Concurrency result in sharing of resources result in problem like deadlocks & resource starvation.

⊙



remainder section (lock=0)

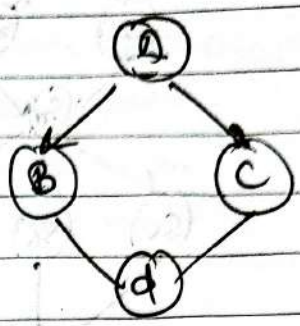
(i) mutual exclusion:- only single process run at a time.

(ii) progress:- every process get chance to run in a critical section.

(iii) Bound and wait:-

Implementation of concurrency through parbegin/ parend :-

=>



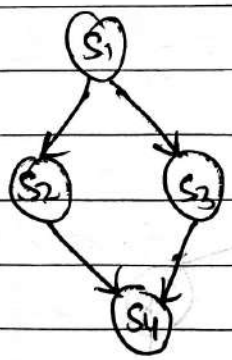
Ex1.

⊙ code :-

```
begin
    S1;
    S2;
    S3;
end
```

OR `begin S1; S2, S3 end`

Ex2

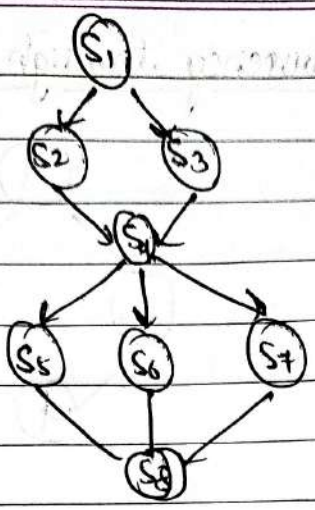


code :-

```
begin
S1;
parbegin
    S2;
    S3;
parend;
end
```

→ Interleaved.

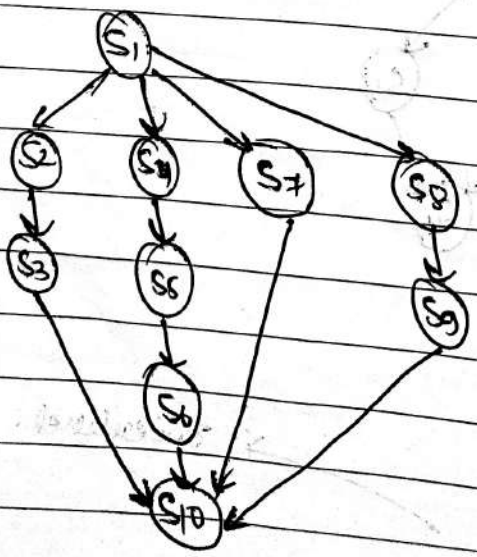
Ex3:



Code:-

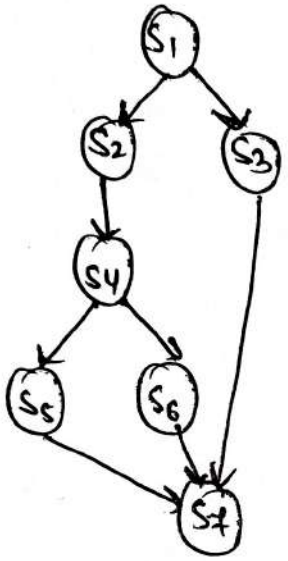
```
begin
  S1;
  parbegin
    S2;
    S3;
  parend
  S4;
  Parbegin
    S5;
    S6;
    S7;
  Parend
  S8;
end
```

Ex4:



```
    S1;
    parbegin
        begin S2, S3 end
        begin S4, S5, S6 end
        S7;
        begin S8, S9 end
    parend
    S10;
end
```

Ex 5:-



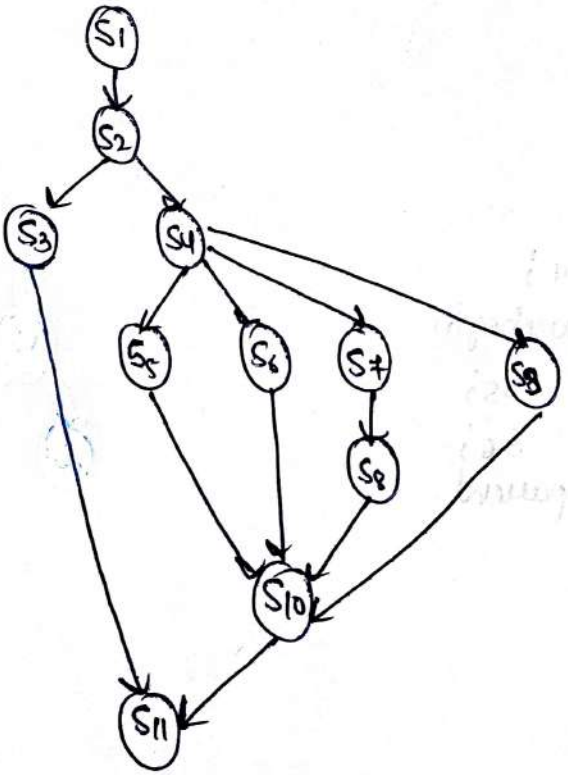
Code:-

```
begin
    S1;
    parbegin
        begin S2;
              S4;
              parbegin
                  S5;
                  S6;
              parend
        end
        S3;
    parend
    S7;
end.
```

Ex6:-

```
begin
    S1;
    S2;
    parbegin
        S3;
        begin
            S4;
            parbegin
                S5;
                S6;
                begin
                    S7;
                    S8;
                end
                S9;
            parend
            S10;
        end
    parend
    S11
end
```

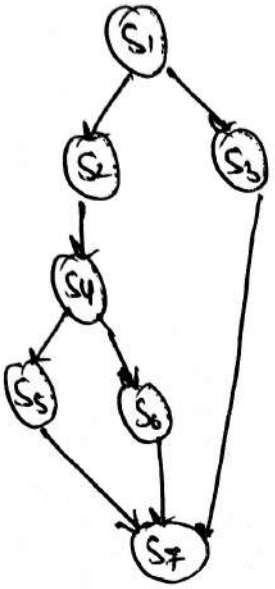
Graph :-



⊙ Fork & Join:-

→ fork split into two
→ join two all merge

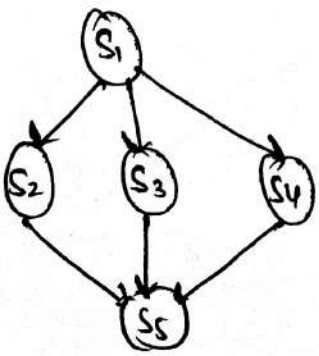
Qu1:



Code:

```
Count = 3
 S1
 Fork L1
   S3  go to L3
L1; S2
    S4
    Fork L2
      S6  go to L3
L2; S5
L3; Join count
    S7
```

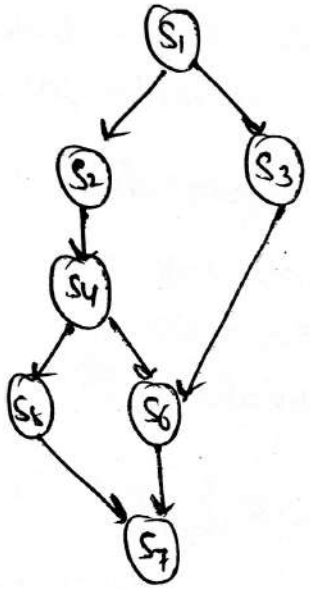
Qu2.



Code:-

```
Count = 3
    S1
    Fork L1
    S4
    go to L3
L1: Fork L2
    S3  go to L3
L2: S2
L3: Join count;
    S5;
```

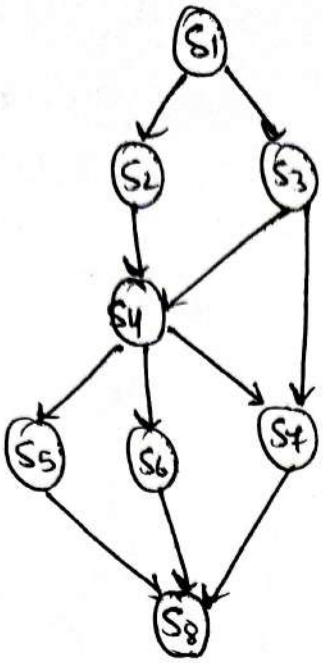
Que3:



Code:-

```
Count = 2
  S1:
  Fork L1
  S3  go to L2
L1: S2
  : S4
    Fork L2
      S5  go to L3
L2: Join C1
    S6
L3: Join C2
    S7;
```

Q4



Code:-

$C_1 = 2$, $C_2 = 2$, $C_3 = 3$

S_1

Fork L1

S_2 go to L

L1: S_3

S_4

Fork

⊙ Inter-process communication (IPC):-

⊙ Interprocess communication is a mechanism which allows processes to communicate & synchronize their actions.

There are two types of process in the system:-
(i) Independent.
(ii) Dependent.

- Two or more processes are said to be dependent if they are affected by each other like producer-consumer, dining philosopher, reader-writer etc.
- Dependent process also called as co-operating, coordinating or communicating processes.

Advantages of dependent process
(i) Information sharing.
(ii) Computational speedup.
(iii) Modularity.

IPC methods:- (Models):-

(i) Message Passing.
(ii) shared memory.
(iii) Pipe().
(iv) socket().

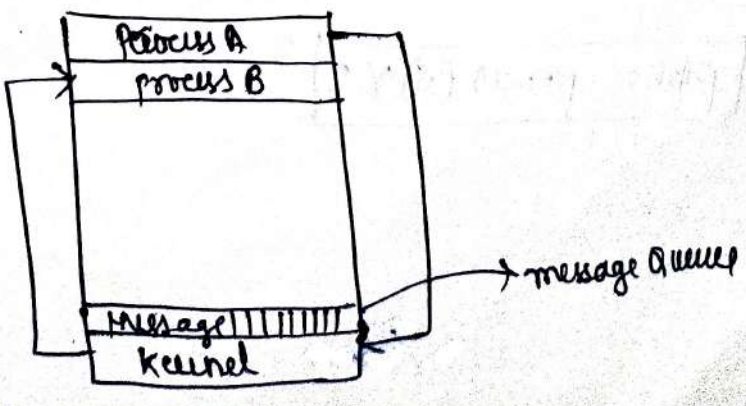
⊙ Message passing:-

⊙ process A → message → communicate → process B

kernel

⊙ Advantages:-

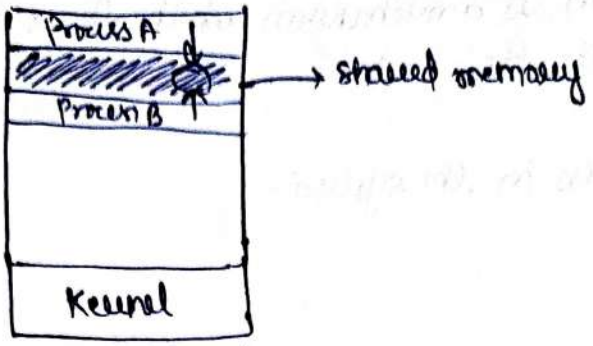
(i) Simplicity.
(ii) No synchronization.
(iii) Distributed system friendly.



⊙ Disadvantages:- (i) Slower comm.

⊙ shared memory:-

⊙



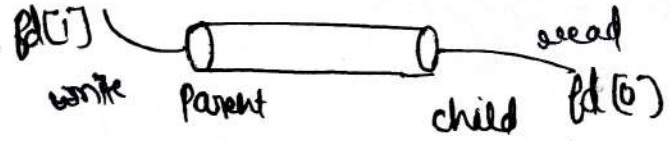
Advantages	Disadvantages
(i) Fast communication.	(i) Complex Synchronization.
(ii) Efficient for large data transfer.	(ii) Security Risks.
(iii) Less kernel Involvement	(iii) Limited to a single Machine.

⊙ pipe:-

⊙ oldest IPC model originally used in UNIX but made even more powerful in windows.

→ pipes are in-core files reside in main memory instead on disk, as other global data structures stack, queue etc.

→ only one way communication is possible in a pipe single one can write and other only can read.



two types of pipe: (in windows):-

(i) Named.

(ii) Unnamed.

⇒ pipe = process (n) X 2

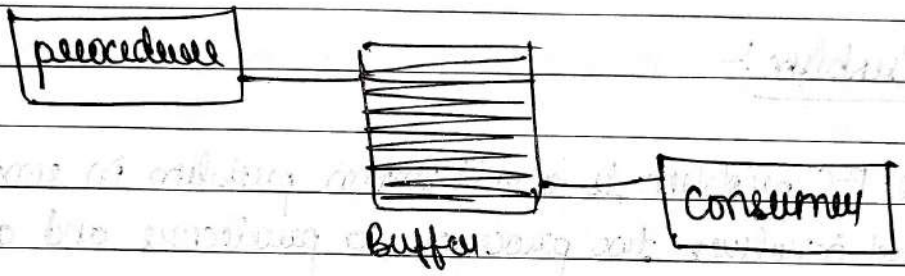
PRODUCER-CONSUMER

Date / /
Page No.
Shivalal

Producer-consumer problem:-

The producer-consumer problem is a classical multi-process synchronization problem, that is we are trying to achieve synchronization between more than one processes.

Note:- only cooperative process



procedure:-

```
int count = 0;
void producer(void){
    int itemp;
    while(1){
        produce_item(itemp)
        while(count==n);
        buffer[in] = itemp;
        in = (in+1) mod n
        count = count+1;
    }
}
```

→ Load Rp, m[count]
Incr Rp;
Store m[count], Rp;

Consumer

```
int count;
void Consumer(void){
```

```
int itemC;
while (1) {
    while (count == 0);
    itemC = buffer [out];
    out = (out+1) mod n;
    count = count - 1;
}
}
```

→ load RC, m[count]
Decr RC
Store m[count], RC.

⊙ Problem :-

⊙ The P-C problem is a well known problem in concurrent program that involves two processes, a producure and a consumer, that share a buffer are queue.

→ producer produce and put it into the buffer. (data).

→ consumer consume and remove from the buffer (data).

⊙ so they need to work in synchronize to avoid making or taking too much at once.

Case :- Producer (I_1, I_2) consumer (I_1 I_2) producer (I_3) consumer (I_3)

→ flow → flow →

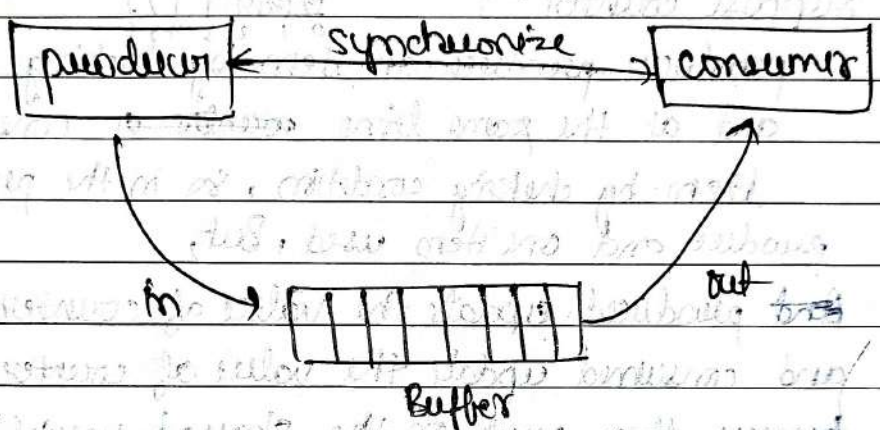
OR

Date / /
Page No.
Shivalal

producer - consumer problem:-

producer - process produce information that is consumed by the consumer process.

→ producer: need to cause → (buffer full):
→ consumer: need to cause → (buffer empty)



producer:-

```
while (true){
    // produce an item
    while (counter == buffer size);
        // do nothing
    buffer [in] = next produced;
    in = (in+1) % buffer size;
        counter ++;
}
```

Consumer:-

```
while (true){
    while (counter = 0);
        // do nothing;
```

```
next consumed = buffer [out]
out = (out + 1) % buffer size;
counter --;
// consume the item
}
```

problem condition:-

⊙ Suppose counter = 4

→ producer produce an item by checking condition and at the same time consumer consume the item by checking condition, so in the process one item produce and one item used, But,

produced update the value of counter to 5.
and consumed update the value of counter to 3
because they work on the shared variable at the same tim
But the write value is 4, so, this is the problem

Note:- The solution of this is to allow only one process to work on the shared variable.

⊙ Solution of P-C problem:-

⊙ semaphore — signal (inc.)
— wait (dec.)

⊙ producer:-

Semaphore empty = n
Semaphore full = 0
semaphore mutex = 1 ...

```
void producer(){
    while(true){
        // produce an item
        wait(Empty);
        wait(mutex);
        add();
        signal(mutex);
        signal(full);
    }
}
```

Consumer:-

→ Semaphore empty = n
→ Semaphore full = 0
→ Semaphore mutex = 1

```
void consumer(){
    while(true){
        // consume an item
        wait(full);
        wait(mutex);
        consume();
        signal(mutex);
        signal(Empty);
    }
}
```

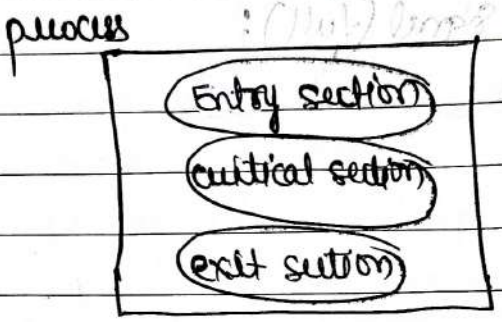
The solution of the producer-consumer problem is to use controls like locks, semaphores. The producer waits if the buffer is full and the consumer waits if the buffer is empty. This ensures they don't interfere with each other, keeping everything in balance.

CRITICAL SECTION

Date / /
Page No.
Shivalal

⊙ Critical section problem:-

⊙ The critical problem in OS occures when multiple processes need to access and modify shared resources concurrently. It ensures that only one process accesses the critical section at a time to prevent data inconsistency.



⊙ Solution:-

Four condition.

primary
1. Mutual Exclusion.
2. progress.

secondary
3. Bounded wait.
4. No assumption related to Hardware, speed.

DEKER'S

Date / /
Page No.
Shivalal

⊙ Deker's Algorithm for critical section problem solution:-

process (I)

```
do{
    flag[i] = true;
    while (flag[j]){
        If (turn == j){
            flag[i] = false;
            while (turn == j);
            flag[i] = true;
        }
    }
    critical section
    turn = = j;
        flag[i] = false;
    remainder section
} while (true)
```

process (J)

```
do{
    flag[j] = true;
    while (flag[i]){
        If (turn == i){
            flag[j] = false;
            while (turn = i);
            flag[j] = true;
        }
    }
    critical section
    turn == i;
        flag[j] = false;
    remainder section
} while(true)
```

PETERSON'S

Date / /
Page No.
Shivalal

① Peterson's solution :-

(I) (II)

```
Boolean flag[2];
int turn;
flag[0] = flag[1] = false
```

```
do{
  flag[i] = true;
  turn = j;
  while (flag[j] & turn = j);
    critical section
  flag[i] = false;
    remainder section
}
  while (1);
```

```
do {
  flag[j] = true;
  turn = i;
  while ( flag[i] & turn =
    critical section
  flag[j] = false;
    remainder section
}
  while(1);
```

⊙ **Semaphore solution:-**

⊙ A semaphore is a integer variable which is used to restrict access to shared resources.

→ It is a synchronization tool which is used to deal with critical section problem.

→ when a process wants to enter in critical section it need to take permission from semaphore first.

- wait(S) (dec.)
- signal(S) (inc.)

⊙

```
wait (S)
{
 while (S<=0);
 S=S-1;
}
```

```
signal(S)
{
 S=S+1;
}
```

⇒ Let P_1, P_2, P_3, and ... P_n are the process that wants to go to the critical section

first S=1

```
do {
 wait(S);
 // critical section
 signal(S);
 // remainder section.
}
 while (True)
```

Date / /
Page No.
Shivalal

⊙ Synchronization hardware solution:-

⊙ : In starting L=0 (L = Lock)

```
while (lock == 1);   ] entry code
lock = 1.
// critical section
lock = 0             ← exit code
```

Test and set instruction:-

```
while ( test_and_set(& lock));
// critical section
lock = false;
```

←——————→

```
boolean test_and_set (boolean * target)
{
    boolean rv = * target;
    * target = true;
    return rv;
}
```

⊙ **DINING PHILOSPHER** (problem)

⊙ **READERS-WRITERS** (problem)

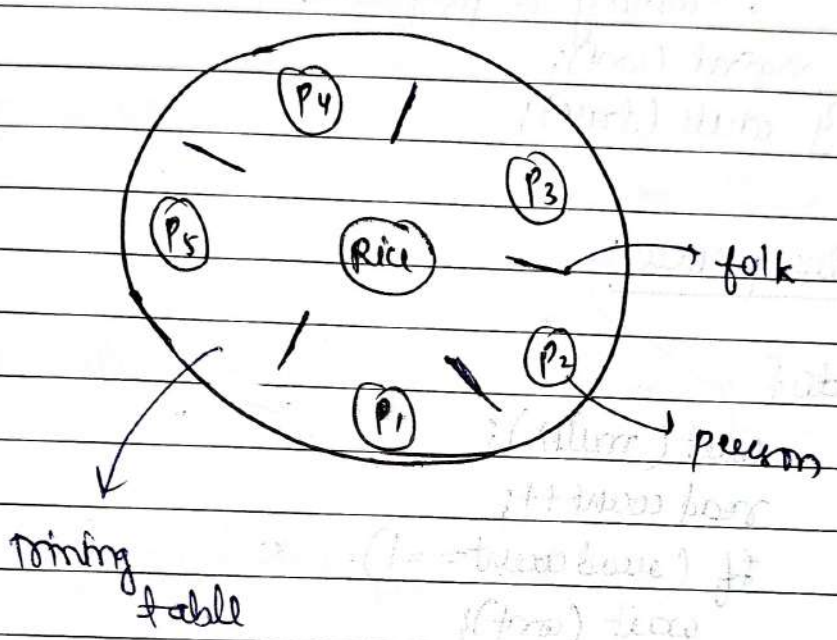
——→

Date / /
Page No.
Shivalal

Classical problem in concurrency:-

⊙ Dining philospher problem:-

⊙ The Dining philospher problem is a classic synchronization problem in computer science that involves multiple processes (philosphers) shared a limited set of resources (forks) in order to perform a task (eating).



Solution Maximum 2 person's can eat at the same time.

⊙ Readers-writers problem:-

⊙ The readers-writers problem manages shared data so multiple readers can read at the same time, but only one writer can write, ensuring no conflicts.

⊙ Solution using semphore:-

semaphore mutex = 1
Semaphore wrt = 1
int readcount = 0

⊙ writers process :-

```
do {
    wait(wrt);
    /* writing is performed */
    signal(wrt);
} while(true);
```

⊙ Reader process :-

```
do {
    wait(mutex);
    read count++;
    if (read count == 1)
        wait(wrt);
    signal(mutex);
    /* Reading is performed */
    wait(mutex);
    read count--;
    if (read count == 0)
        signal(wrt);
        signal(mutex);
    }
    while(true)
```

Date 14-10-24

DEADLOCK

Date / /
Page No.
Shivalal

⊙ **Deadlock:-**

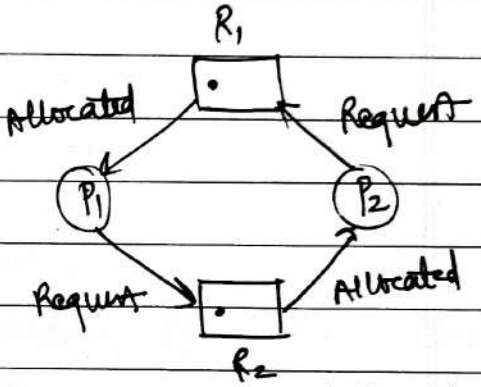
⊙ Deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.

OR.

If too or more processes are waiting on happening of some event, which never happens, then we say these process are involed in deadlock.

⊙ **Deadlock conditions:-**

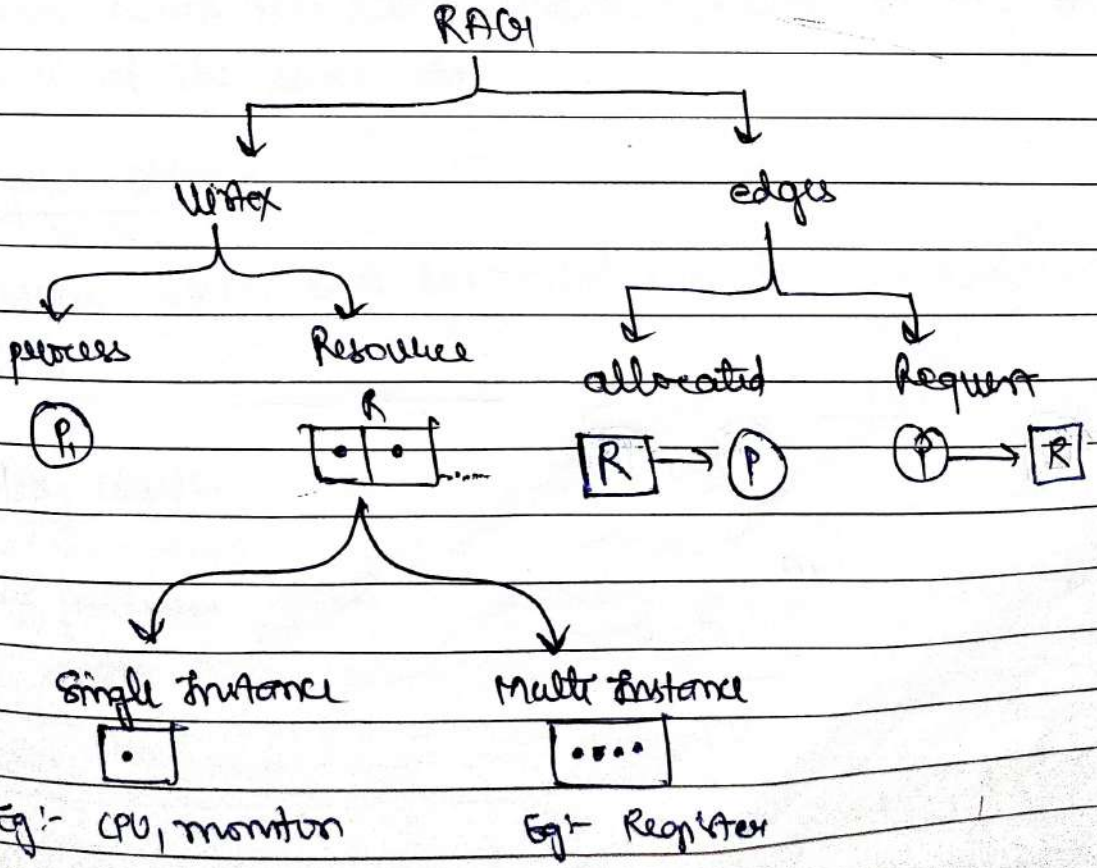
(i) Mutual Exclusive
(ii) Hold & wait.
(iii) No-preemption
(iv) Circular wait.



∴ Deadlock

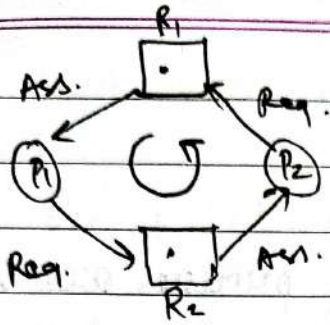
⊙ **Resource Allocation Graph:- (RAG):-**

⊙



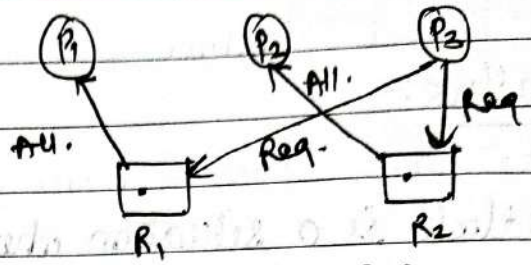
Date / /
Page No.
Shivalal

(i)



∴ Single Instance

(ii)



∴ Single Instance.

Ans (i) Deadlock iteration Ans (ii). No Deadlock occur.

Necessary condition for deadlocks:-

(i) Mutual Exclusion:-

- If two process cannot use same resource at same time.

(ii) Hold and wait:-

- A process waits for some resources while holding and another resource at the same time.

(iii) No-preemption:-

- The process which once scheduled will be executed till the completion.

(iv) Circular wait:-

- All the processes must be waiting for the resources in a cyclic manner.

Various method to handle deadlocks:-

(i) Deadlock ignorance (ostrich method):-

⊙ Deadlock ignorance means ignoring deadlocks, assuming they are rare or won't happen, and not adding any mechanism to prevent or resolve them.

(ii) Deadlock prevention:-

⊙ Deadlock prevention ensures a system avoids deadlocks by eliminating one of the four necessary condition: mutual exclusion, hold and wait, no-preemption, or circular wait.

1. Eliminating mutual exclusion:- Share resource wherever possible.
2. Avoiding hold and wait:- Request all resource at once.
3. Allowing preemption:- Take resource away from processes if needed.
4. Breaking circular wait:- Impose a fixed order for resource requests.

(iii) Deadlock detection and recovery:-

⊙ Involves identifying processes in a system that are in a deadlock state by checking resources allocation and process dependencies. Algorithm like the wait-for Graph (WFG) or Banker's Algorithm are commonly used.

⊙ Once detected, recovery involves breaking the deadlock by:

(i) Process Termination:- Kill one or more processes involved in the deadlock.

(ii) Resource preemption:- Forcibly take resources from processes and reallocate them.

(iv) Dead avoidance:-

⊙ Banker's Algorithm:-

Date 20/10/24

METHOD (Banker's Method)

Date / /
Page No.
Shivalal

⊙ Banker's Method :-

Qu: Consider a system with 5 process P_0 to P_4 and three resource of type A, B & C resource A → 10, B → 5, C → 7 suppose at the time T_0.

process	Allocation A B C	Max A B C	Available A B C	need A B C
P_0	0 1 0	7 5 3	10 5 7	7 4 3
P_1	2 0 0	3 2 2		1 2 2
P_2	3 0 2	9 0 2		6 0 0
P_3	2 1 1	2 2 2		0 1 1
P_4	0 0 2	4 3 3		4 3 1

Sol^n:

Start

A	B	C
10	5	7

Now:- Available

A	B	C
3	3	2

P_1 ↓ P_3 ↓ P_4 ↓ P_0 ↓ P_2

$= 3\ 3\ 2$
$+\ 2\ 0\ 0$
$= 5\ 3\ 2$
$+\ 2\ 1\ 1$
$= 7\ 4\ 3$
$+\ 0\ 0\ 2$
$= 7\ 4\ 5$
$+\ 0\ 1\ 0$
$= 7\ 5\ 5$
$+\ 3\ 0\ 2$
$= 10\ 5\ 7$

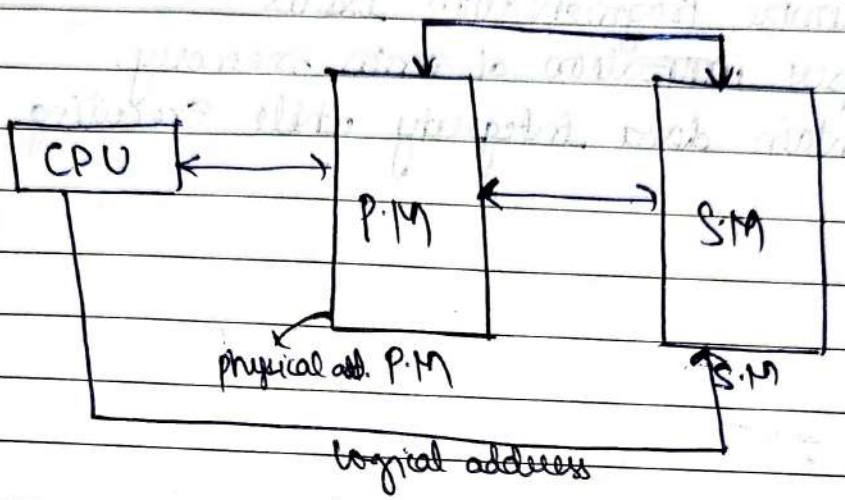
2) Safe Sequence = $P_1 \rightarrow P_3 \rightarrow P_4 \rightarrow P_0 \rightarrow P_2$.

Date / /
Page No.
Shivalal

Memory Management.....

⊙ Memory management:-

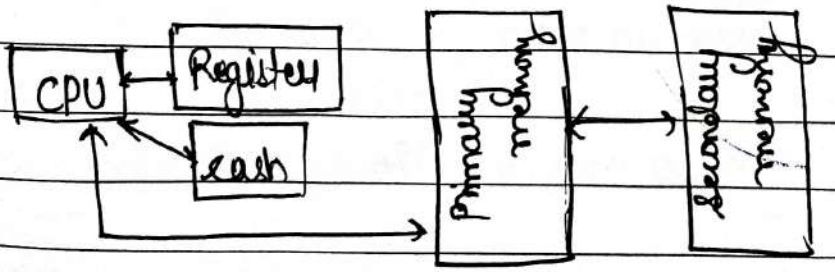
(i) primary memory
(ii) Secondary memory.



⊙ Memory management is a form of resource management applied to computer memory.

→ The essential requirement of memory management is to provide the ways to dynamically portions of memory to programs at their request, and free it for reuse when no longer needed.

⊙ Memory management → method of managing primary memory



Goal:- Efficient utilization of memory.

Date / /
Page No.
Shivalal

⊙ Why memory management is important?

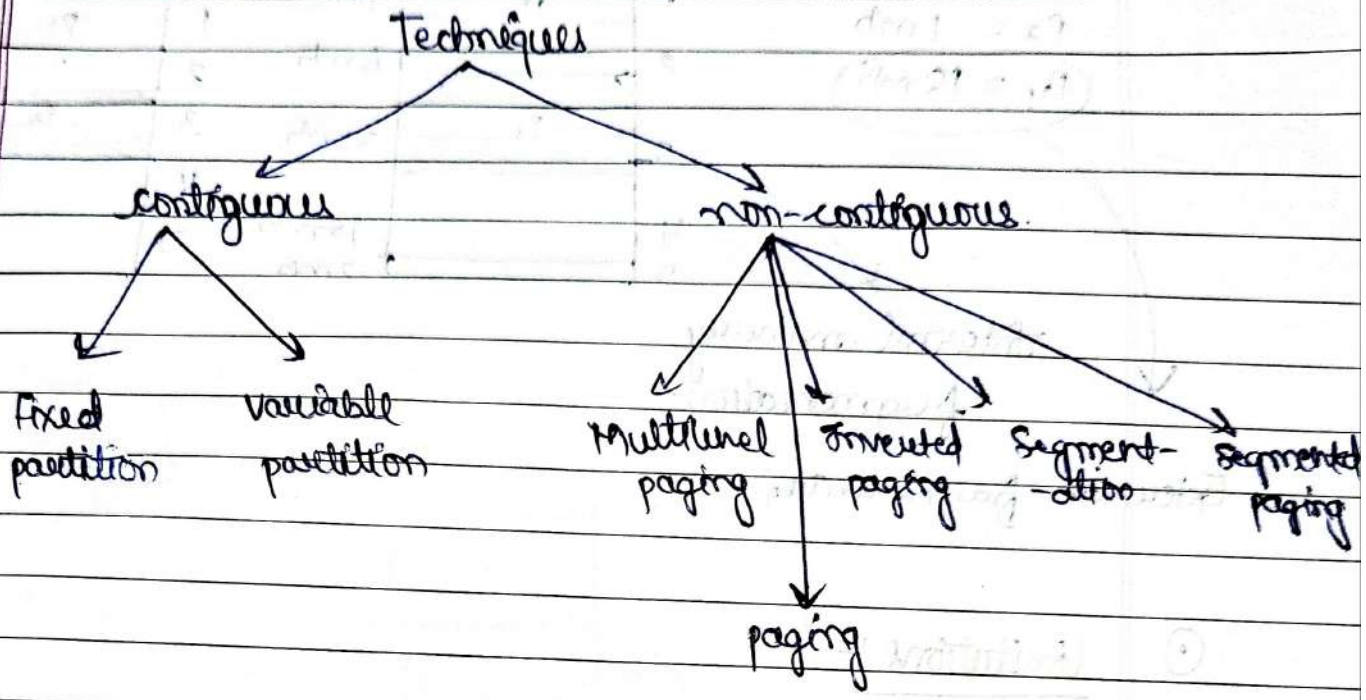
(i) Allocate and de-allocate memory before and after process Execution.
(ii) To keep track of used memory space by processes.
(iii) To minimize fragmentation issues.
(iv) To proper utilization of main memory.
(v) To maintain data integrity while executing of process

Date 05/11/24

TECHNIQUES

Date / /
Page No.
Shivalal

⊙ Memory management Techniques:-



(i) contiguous.
(ii) non-contiguous.

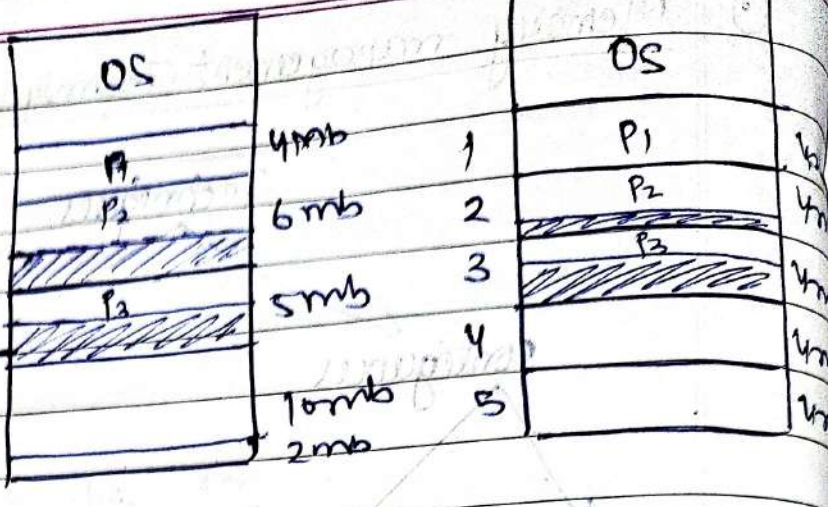
⊙ Contiguous:- Contiguous memory allocation is when a program is given one large, unbroken block of memory, with all the memory addresses next to each other. This makes accessing memory faster, but it can lead to wasted space if there isn't enough room for the program.

(i) Fixed partitioning:-

⊙
→ No. of partitions are fixed.
→ Size of each partition may or may not same.
→ Contiguous allocation
→ This technique is also known as a static partitioning.

Multiprogramming with fixed partitions:-

$P_1 = 4mb$
$P_2 = 3mb$
$P_3 = 1mb$
$P_4 = 18mb$



Internal memory fragmentation.

External fragmentation

⊙ Limitations :-

1. Internal fragmentation.
2. Limit in process size.
3. Limitation on degree of multiprogramming.
4. External fragmentation.

⊙ Internal fragmentation :-

⊙ Internal fragmentation in an operating system occurs when memory is allocated in fixed-sized blocks, and the allocated block is larger than the required memory. This leaves unused space within the block, which cannot be utilized by other processes, leading to memory wastage.

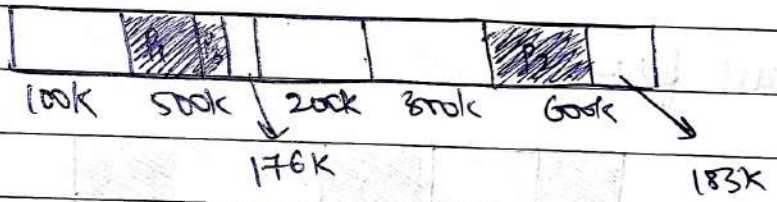
⊙ **Contiguous allocation:-** In this we trying to solve a problem by satisfying the request of process which demands some memories.

(i) First Fit.
(ii) Best Fit.
(iii) Worst Fit.

Ques:

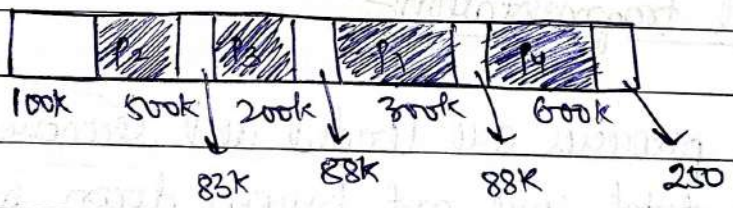
$P_1 = 212K$
$P_2 = 417K$
$P_3 = 112K$
$P_4 = 350K$

→ First fit:-

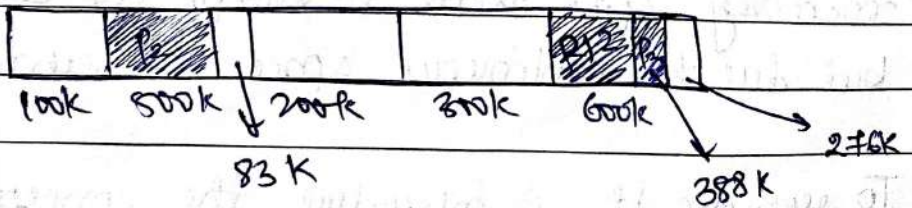


P_4 → not allocated.

→ Best Fit:-

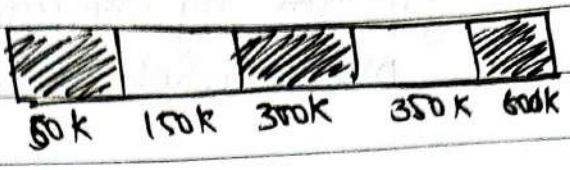


→ Worst Fit:-

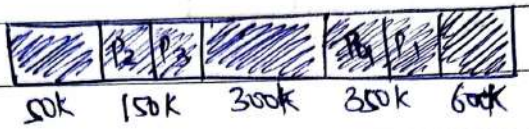


P_4 → not allocated

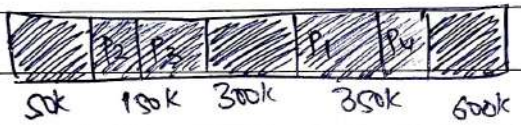
Q.2 Consider the status of memory



(i) First fit:-



(ii) Best fit:-



(iii) worst fit:-

⊙ External fragmentation:-

⊙ When the processes are loaded and removed from the mem and the total space get broken down into no. of blocks so, External fragmentation is the condition when total memory space exists to satisfy the request is sufficient but due to continuous space we cannot use it.

→ To remove it we introduce the concept of Fixed sized partitioning and paging, either the complete block is allocated or nothing is allocated.

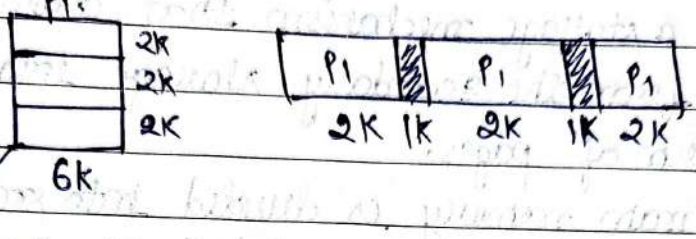
Date 06/10/24

PAGING.....

Date / /
Page No.
Shivalal

⊙ Why paging is needed :-

⇒ Let, suppose

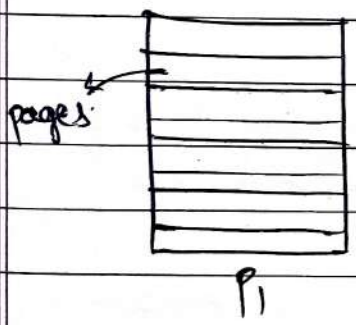


divide into 3 parts (non-contigous)

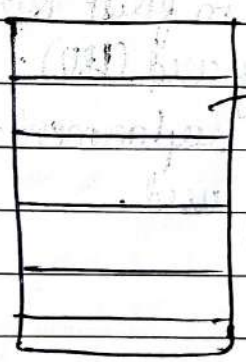
⇒ Although, there is 6K space available in the main memory in the form of 2K holes but that remains useless until it become contigeous. This is a serious problem to address.

⇒ To solve this we use the concept of paging.

⊙



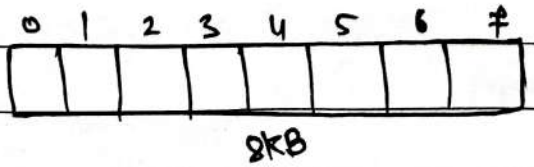
Divide progess into pages.

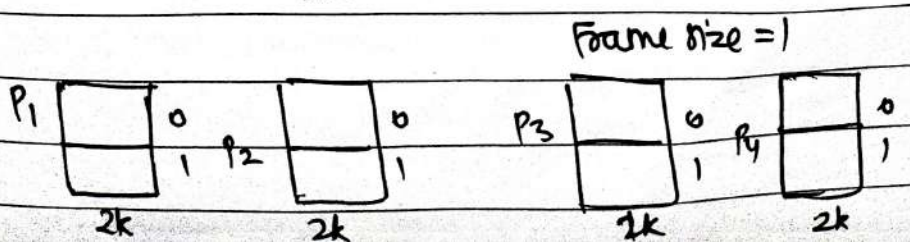


Divide Memory into frames

Page size = Frame size

Example :-





Date / /
Page No.
Shivalal

⇒ we divide memory and process, by this we can easily allocate them.

⊙ Paging:-

⊙ paging is a storage mechanism that allows OS to retrieve processes from the secondary storage into the main memory in the form of pages.

The main memory is divided into small fixed-size blocks of physical memory, which is called frames.

→ Pages size = Frame size, to avoid external fragmentation

⊙ Paging (page) replacement algorithms:-

1. FIFO (First in first out).
2. Least recently used (LRU).
3. optimal Page replacement.
4. Most recently used.

Date 06/11/24

FIFO

Date / /
Page No.
Shivlal

* **FIFO (first in first out):-**

⊙ Consider a system which can allow three frames at a time the following string of pages are suffered.

⊙ Example:-

Ques

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	4	4	4	5	5	5	5	5	5
	2	2	2	1	1	1	1	1	3	3	3
	3	3	3	3	2	2	2	2	2	4	4
							★	★			★

★ → page hit.

Ques.

7	0	1	2	0	3	0	4	2	3	0	3	1	2	0
7	7	7	2	2	2	2	4	4	4	0	0	0	0	0
	0	0	0	0	3	3	3	2	2	2	2	1	1	1
		1	1	1	1	0	0	0	3	3	3	3	2	2
				★							★			★

★ → page hit.

- page Hit Ratio = $\frac{\text{No. of hits}}{\text{No. of Reference}} = \frac{3}{15} = \frac{1}{5} \times 100 = 20\%$

- page fault Ratio = $\frac{\text{No. of Page fault}}{\text{No. of Reference}} = \frac{12}{15} \times \frac{20}{100} = 80\%$

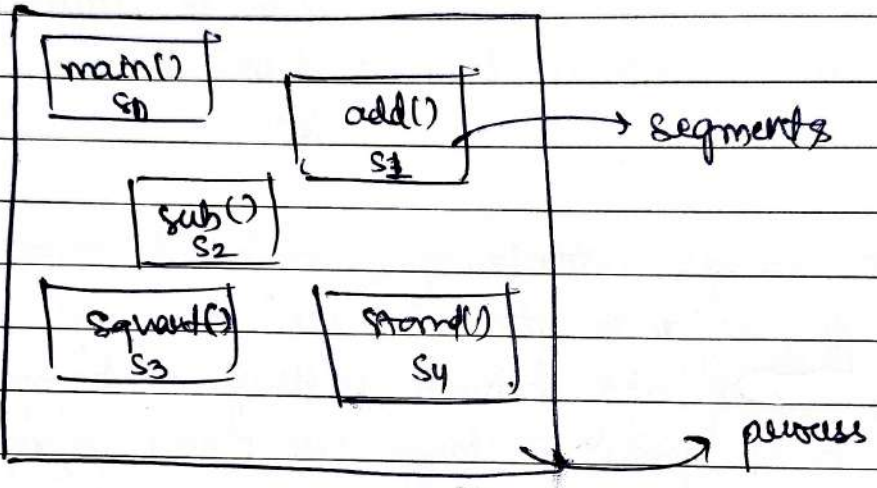
Date 12/11/24

SEGMENTATION

Date / /
Page No.
Shivalal

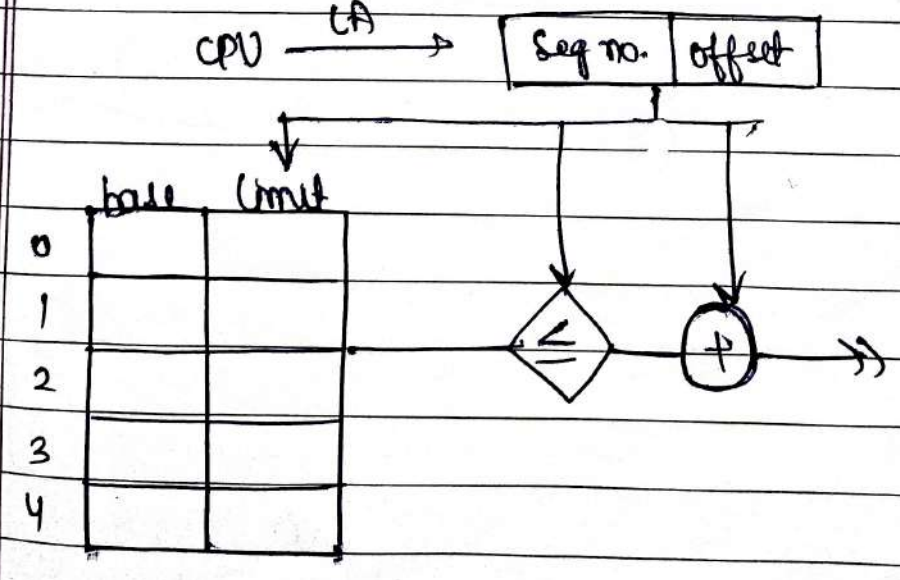
Segmentation:

- Segmentation is a memory management technique in which the memory is divided into the variable size parts. Each part is known as a segment which can be allowed to a process.



→ pages are of same size but segments are may or may not be same size.

→ CPU generates a LA (logical address).



Date 25-11-24

PAGED SEGMENTATION

Date / /
Page No.
Shivalal

⊙ Paging with Segmentation :-

(i) Process $\xrightarrow{\text{divide}}$ Segments $\xrightarrow{\text{divides}}$ Pages $\xrightarrow{\text{stored}}$ main memory

⊙ It contains both paging & segmentation schemes to get best features of both.
In this scheme, each segment is divided into pages.

OR

→ In a combined paging segmentation system a user's address space is divided into number of segments. Each segment is further divided into numbers of fixed sized pages which are equal in length to a main memory frame.

→ Segmentation is visible to the programmer or it support user's view.

→ paging is transparent to the programmer.

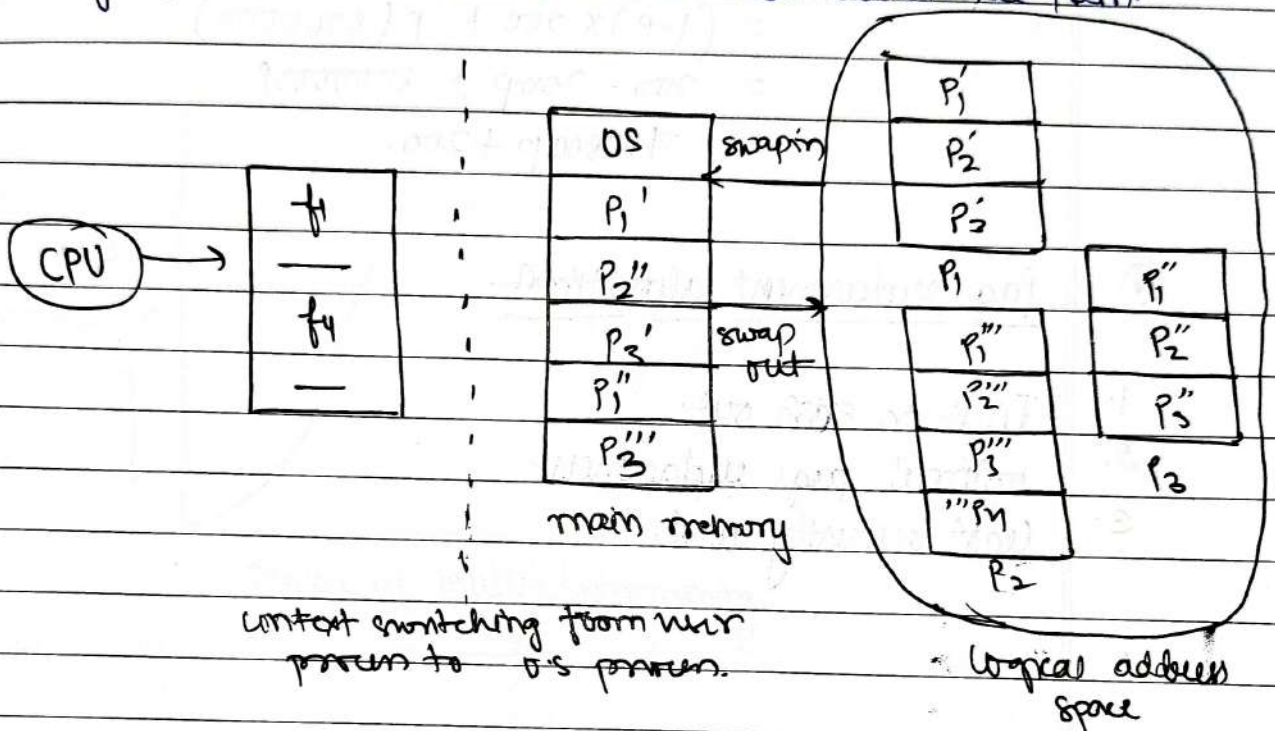
Date 27-11-24

VIRTUAL memory

Date / /
Page No.
Shivalal

⊙ Virtual memory concept:-

⊙ Virtual memory in an operating system is a memory management technique that allows a computer to use more memory than physically available by using a portion of the storage (like a hard drive or SSD) as additional RAM.



→ Effective memory Access time

$$EMAT = p\,(\text{page fault service time}) + (1-p)\,(\text{main memory access (in nsec)})$$

→ where, p is page fault or probability of page fault.
$(0\% \le p \le 100\%)$

⊙ Demand paging:-

⊙ Demand paging is a virtual memory technique where pages are loaded into RAM only when a program tries to access them, reducing memory usage and speeding up execution.

Date / / Page No. Shivalal

⊙ Performance of demand paging :-

Page fault :- page is not present in main memory (RAM)

Qu!. Memory Access time = 200 nano sec
avg. page fault service time = 8 milsec.
EAT = (1-p) x 200 + p (8 m sec)
= (1-p) x 200 + p (8000000)
= 200 - 200p + 8000000p
= 799800 p + 200.

⊙ Page replacement algorithms

1. First in First out
2. optimal page replacement
3. least recently used.

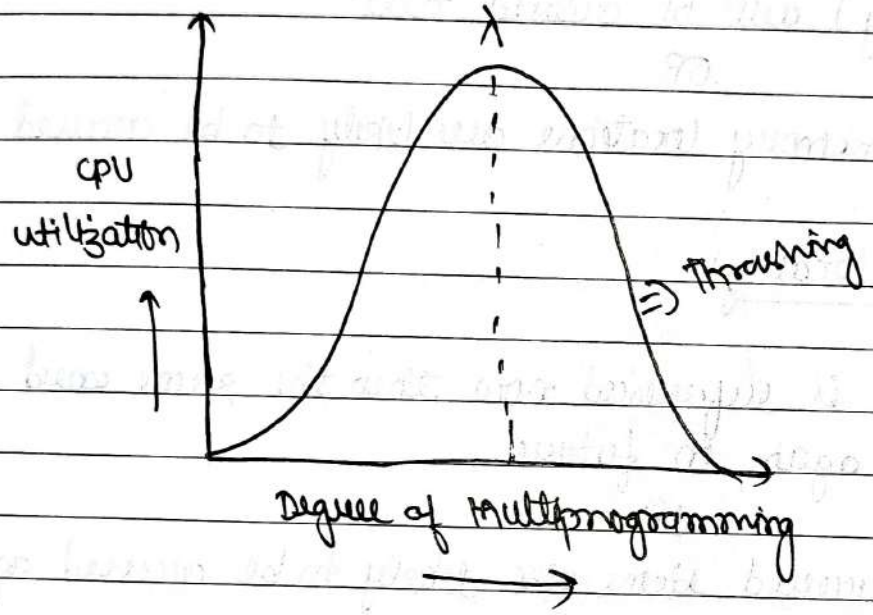
Date 27-11-24

THRASHING

Date / /
Page No.
Shivalal

⊙ Thrashing :-

⊙ Thrashing in os is a phenomena that occurs in computer operating system when the system spend an excessive amount of time on swapping data between physical memory (RAM) and virtual memory (disk storage) due to high memory demanded and low available resources.



How to reduce thrashing :-

(i) Increase RAM.
(ii) Optimize multiprogramming.
(iii) Priority scheduling
(iv) Efficient page replacement

⊙ Locality of reference :-

⊙ Locality of reference in os refers to the tendency of a program to access a small set of memory locations repeatedly over a short period.

Date / /
Page No.
Shivalal

Types:

(i) Spatial locality.
(ii) Temporal locality.

⊙ **Spatial locality:-**

⊙ If a word is accessed now then the word adjecent to it (close proximity) will be accessed next.

OR

Nearby memory locations are likely to be accessed soon.

⊙ **Temporal locality:-**

⊙ If a word is referenced now then the same word will be referenced again in future.

OR

Recently accessed items are likely to be accessed again.

⊙ LRU is used in temporal locality.

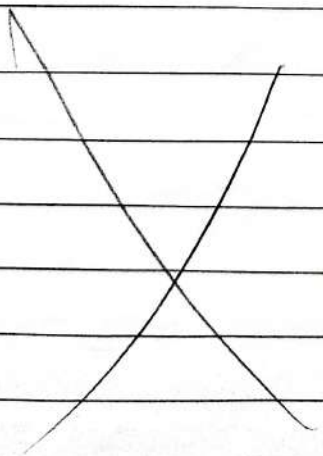
I/O

Date / /
Page No.
Shivalat

- **I/O management and disk scheduling :-**

- I/O management in a OS handles all input and output operations between hardware devices (like keyboard, mouse, disk) and software.

- **Input-output devices :-**



Date 29-11-24

Date / / Page No. Shivalal

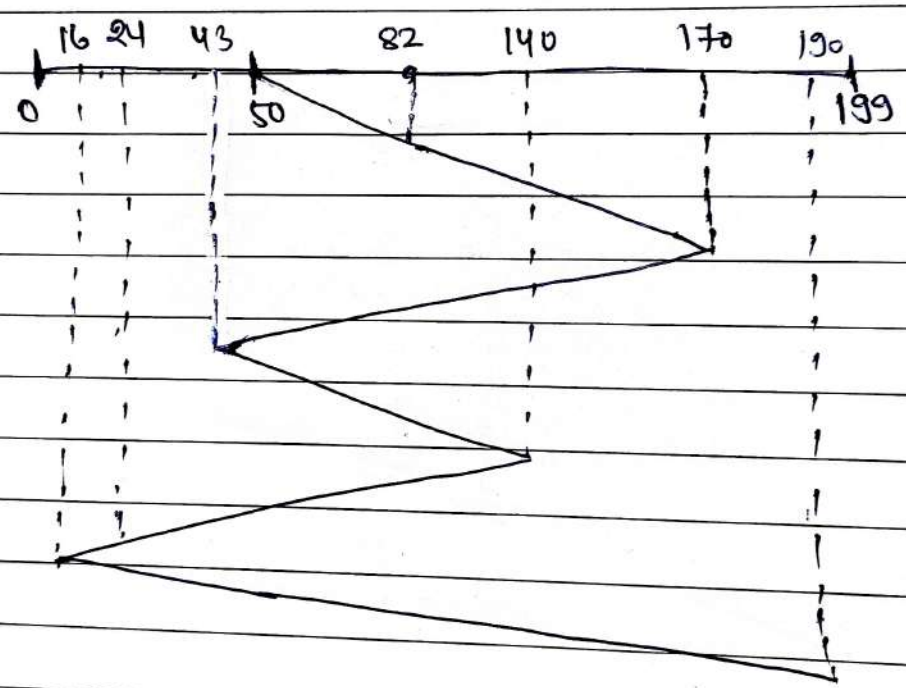
FCFS

⊙ First come first served

→ First come first serve.

Que: A disk contains 200 tracks (0-199) request queue contains track no. 82, 170, 43, 140, 24, 16, 190. respectively current position of read-write head = 50. calculate total no. of tracks movement by read-write head.

Sol:



=) (82-50) + (170-82) + (170-43) + (140-43) + (140-24) + (24-16) + (190-16)

OR

=) (170-50) + (170-43) + (140-43) + (140-16) + (190-16)

=) 642 Ans

⊙ No starvation.

LRU

Date / /
Page No.
Shivalal

* L.R.U (Least recently used):-

⊙ Least Recently used (LRU) is a cache replacement policy in operating system that removes the least recently accessed page from memory to make space for a new page, optimizing memory usage by prioritizing frequently accessed pages.

Qu1.

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0
7	7	7	2	2	2	2	4	4	4	0	0	0	1	1	1	1	1	1
	0	0	0	0	0	0	0	0	3	3	3	3	3	3	0	0	0	0
		1	1	1	3	3	3	2	2	2	2	2	2	2	2	2	7	7
				★		★					★	★		★		★		★

★ → page hit

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0
7	7	7	7	7	3	3	3	3	3	3	3	3	3	3	3	3	7	7
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
		1	1	1	1	1	4	4	4	4	4	4	1	1	1	1	1	1
			2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
				★		★		★	★	★	★	★		★	★	★		★

★ → page fault

Date 11/11/24

Questions

Date / /
Page No.
Shivalal

Q1. Consider a reference string with three frames.

FIFO → with 3

1	2	1	3	7	4	5	6	3	1
1	1	1	1	7	7	7	6	6	6
	2	2	2	2	4	4	4	3	3
			3	3	3	5	5	5	1
		★							

★ → page hit

LRU → with 3.

1	2	1	3	7	4	5	6	3	1
1	1	1	1	1	4	4	4	3	3
	2	2	2	7	7	7	6	6	6
			3	3	3	5	5	5	1
		★							

★ → page hit

LRU → with 4

1	2	1	3	7	4	5	6	3	1
1	1	1	1	1	1	5	5	5	5
	2	2	2	2	4	4	4	4	1
			3	3	3	3	6	6	6
				7	7	7	7	3	3
		★							

★ → page hit

FIFO → with 3

1	2	1	3	7	4	5	6	3	1
1	1	1	1	1	4	4	4	4	1
	2	2	2	2	2	5	5	5	5
			3	3	3	3	6	6	6
				7	7	7	7	3	3
		★							

★ → page hit.

Date / /
Page No.
Shivalal

⊙ Belady's anamoly :-

Que!

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	4	4	4	5	5	5	5	5	5
	2	2	2	1	1	1	1	1	3	3	3
		3	3	3	2	2	2	2	2	4	4
							★	★			★

★ → page hit

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	5	5	5	5	4	4
	2	2	2	2	2	2	1	1	1	1	5
		3	3	3	3	3	3	2	2	2	2
			4	4	4	4	4	4	3	3	3
				★	★						

★ ⟶ page hit

⟶ In this condition increase in frame cause increase in page fault.

* Belady's anamoly is a phenomenon in the F.I.F.O where increase the no. of page frames (memory) can lead to an increase in the number of pagefault, rather than a decrease.

OPTIMAL

Date / /
Page No.
Shivalal

* Optimal page replacement:

⊙ Optimal page replacement is a page replacement algorithm th when memory is full, removes the page that won't be needed for the longest time in the future. It's ideal becaus it results in the fewest page faults, but it's usually theoretical since future page requests aren't known in practice.

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0
7	7	7	7	7	3	3	3	3	3	3	3	3	3	3	3	3	7	7
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
		1	1	1	1	1	4	4	4	4	4	4	1	1	1	1	1	1
			2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
				★		★		★	★	★	★	★		★	★	★		★

★ → page hit

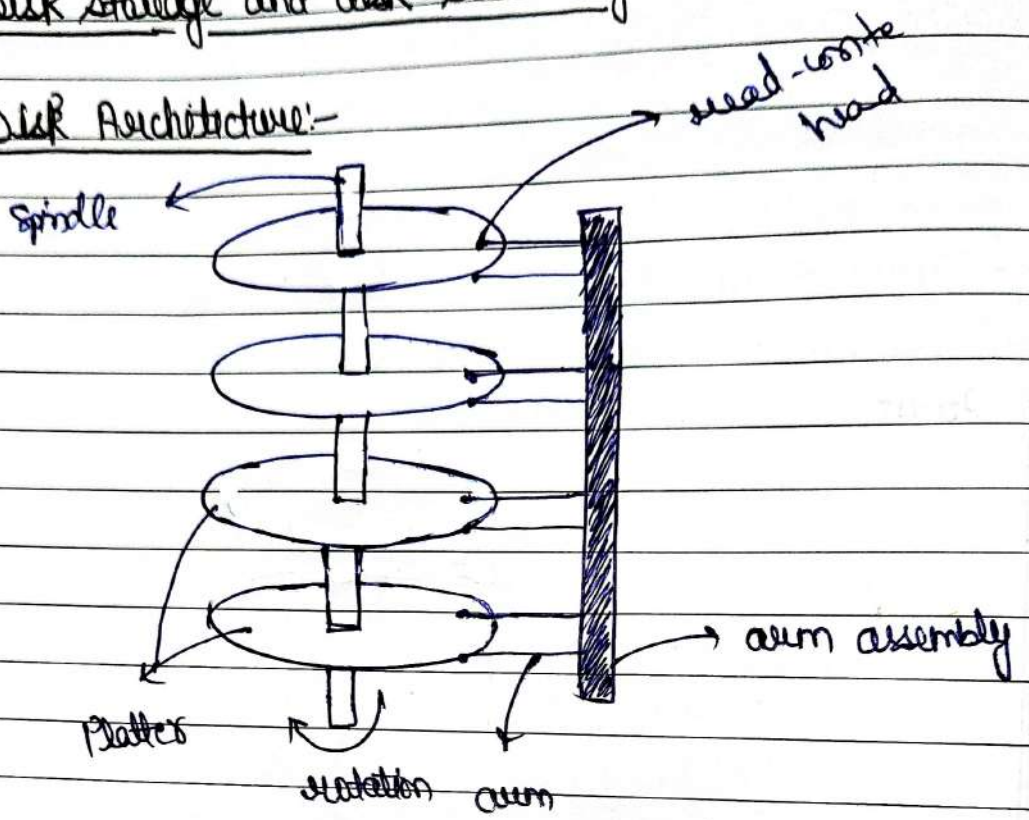
Date 26-11-24

Date / /
Page No.
Shivalal

Disk Scheduling

⊙ Disk storage and disk scheduling:-

Disk Architecture:-



platter → surface → track → sectors → Data.

= 8 platter = 16 surface

= 8X2X256X512 = total no. of sectors in the disk

= Disk size = PXSXTX SectorX data

= 8X2X256X512X 512KB (∵Ex)

= 2^{40} B.

1K = 2^{10}

1M = 2^{20}

1G = 2^{30}

1T = 2^{40}

Date / /
Page No.
Shivalal

Disk Access time :-

(i) Seek time :- Time taken by read-write head to reach desired track.

(ii) Rotation time :- Time taken for one full rotation (360°)

(iii) Rotational latency :- Time taken to reach to desired sector (half of rotation time).

(iv) Transfer time :- $\frac{\text{Data to be transfer}}{\text{Transfer Rate}}$

where, Transfer rate = {no. of Heads × Capacity of one tracks × No. of Rotations in one second}

⊙ Disk scheduling Algorithm :-

Goal :- To minimize the seek time.

(i) FCFS (first come first serve).
(ii) SSTF (shortest seek time first).
(iii) SCAN.
(iv) LOOK
(v) CSCAN (Circular scan)
(vi) CLOOK (Circular Look).

Date
28-1-24

SSTF

Date / /
Page No.
Shivala

⊙ Shortest seek time first

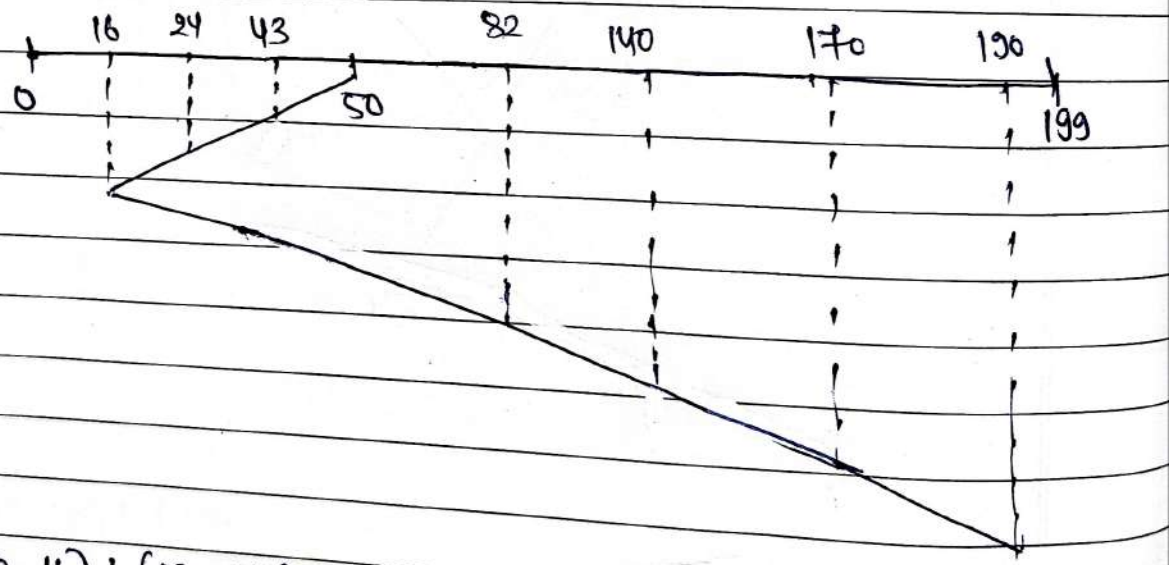
→ shortest seek time first

Que: 200 tracks (0-199), track no, 82, 170, 43, 140, 24, 16, 190, current portion of r-w head = 50.

→ calculate total no. of track movements by R/w head.

→ If R-w head takes 1ms to move from one track to another then total time taken ____ ?

Soln:



⇒ (50-16) + (190-16)

(i) ⇒ 208.

(ii)